

A Transformer-Based Detector for Network Steganography with Improved Generalization



Developed By

Malaika Umbreen	UOC-IT-F2022-87
Laraib Fatima	UOC-IT-F2022-74
Shamama Muzaffar	UOC-IT-F2022-69

Supervised By

Mr. Fahim Abid

Department of Computer Science & Information Technology

Faculty of Computing & Information Technology

The University of Chakwal

Bachelor of Science in Information Technology (2022-26)

DECLARATION

We hereby declare that this project title as **A Transformer-Based Detector for Network Steganography with Improved Generalization**, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software documentation and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Laraib Fatima

Malaika Umbreen

Shamama Muzaffar

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BSIT **A Transformer-Based Detector for Network Steganography with Improved Generalization** has successfully developed and submitted by **Laraib Fatima (UOC-IT-F2022-74)**, **Malaika Umbreen (UOC-IT-F2022-87)** and **Shamama Muzaffar (UOC-IT-F2022-69)** under the supervision of **Mr. Fahim Abid** at Department of Computer Science & Information Technology, The University of Chakwal and that in his opinion; it is fully adequate, in scope, quality and to fulfill the requirement for the degree of Bachelor of Science in Information Technology, Session 2022-26.

Mr. Fahim Abid
Project Supervisor

Dr./Mr./Ms. _____
External Examiner

Ms. Hina Iftikhar
Head of Department

ABSTRACT

Network steganography is one of the most technically challenging cybersecurity threats in which adversaries hide covert data within normal network protocol traffic, making their communications invisible to conventional intrusion detection systems. Existing detection systems built on LSTM, CNN, Random Forest, and SVM fail to generalize to previously unseen steganographic techniques, with documented accuracy drops of over 20 percentage points under cross-technique evaluation.

This project presents a web-based network steganography detection platform powered by the Feature Token Transformer, a Transformer encoder that treats each of 225 flow-level behavioral features as an independent input token. The model simultaneously performs binary detection of covert versus normal traffic, multi-class technique identification across Header Manipulation, Timing Obfuscation, and Flow Blending categories, and protocol attribution across DNS, ICMP, TCP, and UDP protocols. The underlying deep learning model was trained on a hybrid labeled dataset of 13,848 samples constructed from a public Kaggle DNS tunneling dataset and self-generated ICMP, TCP, and UDP covert traffic and self-generated normal traffic. The Leave-One-Technique-Out evaluation protocol yielded 94.42% mean binary detection accuracy, a 0.91% mean false positive rate, outperforming LSTM by 16.02 percentage points, CNN by 18.22 points, Random Forest by 24.62 points, and SVM by 31.32 points. The web application integrates live packet capture via Npcap, real-time covert channel alerts, per-protocol flow analysis, steganography technique visualization, and CSV-based detection log export. The system is deployed as a React.js frontend with a PHP RESTful backend and MySQL database.

ACKNOWLEDGEMENT

All praise is to Almighty Allah who bestowed upon us a minute portion of his boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Mr. Fahim Abid” for his personal supervision, advice, valuable guidance toward the completion of this project. We are deeply indebted to him for encouragement and continual help during this work.

We are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Malaika Umbreen

Laraib Fatima

Shamama Muzaffar

PREFACE

This final year project report has been prepared in fulfillment of the requirements for the Bachelor of Science in Information Technology, session 2022-26 in the academic year 2026.

For preparing of this final year project report, we closely studied the domain of network steganography detection, covert channel analysis, and deep learning-based intrusion detection systems to understand the data requirements and system behaviors needed to address the detection gap documented in the existing literature.

We studied and applied the following tools and technologies: React.js with TypeScript for the web frontend, PHP for the RESTful API backend, MySQL for database management, Python with PyTorch for Transformer model training, Npcap for live network packet capture, and VS Code as the primary development environment.

This project falls within the domains of Machine Learning, Cybersecurity, Web Application Development, and Network Traffic Analysis. The integration of a deep learning model with a real-time web interface was the central engineering challenge of this project.

We are very thankful to our project supervisor “Mr. Fahim Abid” who has helped us to complete this project and we really appreciate all his efforts and support.

We are also very thankful to The University of Chakwal and all the teachers & staff.

ABBREVIATIONS

Abbreviation	Full Form
SRS	Software Requirements Specification
FYP	Final Year Project
AI	Artificial Intelligence
ML	Machine Learning
FR	Functional Requirement
NFR	Non-Functional Requirement
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
SVM	Support Vector Machine
DNS	Domain Name System
ICMP	Internet Control Message Protocol
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
SDLC	Software Development Life Cycle
API	Application Programming Interface
UI	User Interface
LOTO	Leave-One-Technique-Out
SOC	Security Operations Centre
XSS	Cross-Site Scripting
TLS	Transport Layer Security
ERD	Entity Relationship Diagram
CSV	Comma-Separated Values

Table of Contents

Chapter 1: Introduction of the Project	1
1.1. Introduction:	1
1.2. Scope:	3
1.3. Purpose:	5
1.4. Objective:.....	5
1.5. Tools and Technologies:.....	6
1.6. Chapter Summary:	7
Chapter 2: Literature Review & Existing Systems	8
2.1. Literature Review:	8
2.2. Problem Statement:.....	12
2.3. Proposed Solution:.....	13
2.4. Comparative Analysis between Existing and Proposed Solution:.....	13
2.5. Deep Learning Techniques: A Feature Token Transformer	14
2.6. Chapter Summary:.....	19
Chapter 3: Software Requirements Specification	20
3.1. Functional Requirements:	20
3.2. Non-Functional Requirements:.....	21
3.3. Use Case Diagram:	23
3.4. Methodology and Software Lifecycle for This Project:	24
3.5. Chapter Summary:	25
Chapter 4: Design and Architecture	26
4.1. Entity Relationship Diagram (ERD):.....	26
4.2. Sequence Diagrams:	27
4.3. Architecture Design Diagram:	30
4.4. Class Diagram:.....	31
4.5. Database Design:	32
4.6. Interface Design:.....	33
4.7. Chapter Summary:	38
Chapter 5: Software Testing & Implementation	39
5.1. Deriving Test:	39
5.2. Testing Environment:	40
5.3. Testing Techniques:.....	44

5.4. Test Cases:	44
5.5. Chapter Summary:	49
Chapter 6: Conclusion and Future Work.....	50
6.1. Discussion:.....	50
6.2. Conclusion:	51
6.3. Limitations:.....	52
6.4. Future Work:.....	53
References.....	56

List of Figures/Diagrams

Figure 1.1	End-to-end pipeline of the Transformer-Based Network	
	Steganography Detector	2
Figure 1.2	Illustration of covert data transmission methods across an IPv4 network.....	4
Figure 2.1	Standard Transformer encoder-decoder architecture.....	11
Figure 2.2	Dataset Generation Architecture Lab Setup	15
Figure 2.3	Feature Categorization Three-Category Framework.....	15
Figure 2.4	Packet-level to flow-level transformation pipeline.....	16
Figure 2.5	Multi-Protocol Dataset Combination and Preprocessing Pipeline.....	17
Figure 2.6	Preprocessing pipeline of our proposed model	18
Figure 3.1	Use Case Diagram of stegfix.....	23
Figure 3.2	Software Development Lifecycle(SDLC) Phases of stegfix.....	24
Figure 4.1	Entity Relationship Diagram of stegfix.....	26
Figure 4.2	Sequence Diagram:User Authentication Flow	27
Figure 4.3	Sequence Diagram:Live Capture and Detection Flow.....	28
Figure 4.4	Sequence Diagram:CSV Log Export Flow.....	29
Figure 4.5	Architecture Design Diagram of stegfix.....	30
Figure 4.6	Class Diagram of stegfix.....	31
Figure 4.7	Database Design of stegfix.....	32
Figure 4.8	Login Interface Design of stegfix.....	33
Figure 4.9	Dashboard Interface Design of stegfix.....	34
Figure 4.10	Flow Detail Interface of stegfix.....	35
Figure 4.11	Technique Analysis Interface of stegfix.....	35
Figure 4.12	Protocol Analysis Interface of stegfix.....	37
Figure 4.13	Detection Logs Interface.....	37
Figure 5.1	DNS Time Obfuscation Detection.....	41
Figure 5.2	ICMP Header Manipulation Detection.....	42
Figure 5.3	TCP Header Manipulation Detection.....	43
Figure 5.4	UDP Flow Blending Detection.....	43
Figure 5.5	LOTO Evaluation Binary Detection Accuracy and Protocol Accuracy per Split.....	46
Figure 5.6	Binary Detection Confusion Matrices across all three LOTO Splits.....	47
Figure 5.7	Task Accuracy Binary Detection.....	48

List of Tables

Table 2.1	Comparative Analysis between Existing and Proposed Solution	13
Table 5.1	Complete Test Cases and Results.....	44
Table 5.2	FeatureTokenTransformer vs. Baseline Models LOTO Evaluation	45
Table 5.3	LOTO Evaluation Accuracy Per Split	45

Chapter 1: Introduction of the Project

1.1. Introduction:

Billions of packets traverse global networks daily carrying emails, financial transactions, healthcare records, and government communications. While most traffic is legitimate, a growing portion carries hidden content that conventional security tools cannot detect. Modern adversaries conceal communications inside ordinary protocol packets, making their activity invisible to signature-based IDS and statistical anomaly detectors.

Network steganography hides the existence of communication itself not just its contents. An observer sees only normal, protocol-compliant packets. No signature is matched, no threshold crossed, no alert raised.

This project, StegFix, delivers a web-based real-time network steganography detection platform. Its core is the Feature Token Transformer a custom Transformer encoder processing 225 flow-level behavioral features across DNS, ICMP, TCP, and UDP simultaneously. The system identifies covert traffic, classifies the steganographic technique (Header Manipulation, Timing Obfuscation, or Flow Blending), and attributes each detection to the correct protocol all through a browser-accessible dashboard with live packet capture, real-time alerts, and exportable forensic logs.

Each covered protocol offers documented hiding opportunities: DNS encodes data in subdomain query strings (dns2tcp, dnscat2, iodine); ICMP carries hidden data in echo payload and sequence number fields (Ptunnel); TCP exploits initial sequence numbers and flag combinations (Covert_TCP, HCovert); UDP exploits payload content, packet size variation, and inter-arrival timing statistics.

APT groups, state-sponsored actors, and criminal organizations exploit these channels for data exfiltration, command-and-control communication, and long-term covert persistence sustaining hidden access over months without triggering any alert.

Existing approaches fail in compounding ways. Signature-based systems only catch known attacks. Statistical detectors are defeated by evasion techniques. Classical ML

models require protocol-specific manual feature engineering that does not generalize. Deep learning detectors overfit to training distributions and collapse on unseen techniques Cho and Lai documented LSTM accuracy dropping from 99.4% to 76.9% under cross-technique evaluation, a 22.5 percentage point drop that would be operationally catastrophic in a deployed SOC.

The Feature Token Transformer achieves 94.42% mean binary detection accuracy under the Leave-One-Technique-Out protocol, with a minimum floor of 90.60% across all three LOTO splits outperforming LSTM by 16.02 points, CNN by 18.22 points, Random Forest by 24.62 points, and SVM by 31.32 points under identical conditions on the same dataset.

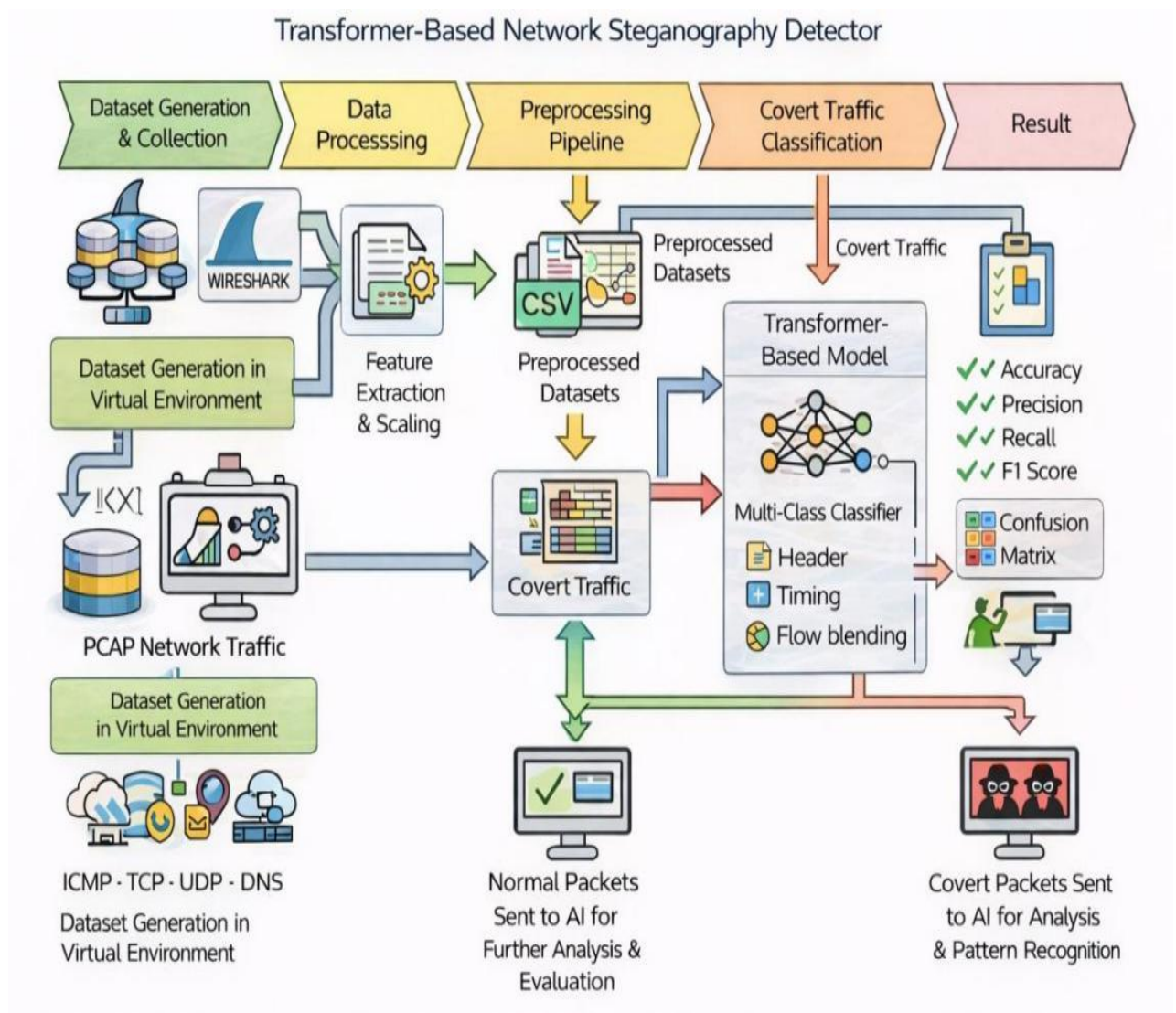


Figure 1.1. End-to-end pipeline of the Transformer-Based Network Steganography Detector

1.2. Scope:

StegFix covers the complete design, implementation, and deployment of a web-based, real-time network steganography detection system operating across four protocols DNS, ICMP, TCP, and UDP by analyzing 225 flow-level behavioral features. The project boundaries are defined below.

What Is Included

- **React.js and TypeScript frontend:** A real-time analyst dashboard with live capture controls, a covert channel alerts panel, a technique distribution chart, a protocol traffic bar graph, a recent detections table, per-protocol Flow Detail Analysis pages, Technique Analysis pages with bar, pie, and line chart visualizations, a Protocol Analysis statistics page, and a Detection Logs page with CSV export.
- **PHP 8.2 RESTful API backend:** Twelve REST endpoints covering user authentication, session management, Npcap capture orchestration, feature vector routing to the model inference endpoint, and MySQL database interaction.
- **Npcap 1.78 live packet capture:** Integrated with the dashboard through Start and Stop controls, capturing source IP, destination IP, protocol, timestamp, and payload attributes without requiring command-line access or specialist configuration.
- **Feature Token Transformer inference microservice:** A pre-trained Transformer encoder served via Python Flask, classifying each flow as Normal or Covert, identifying the steganographic technique used (Header Manipulation, Timing Obfuscation, or Flow Blending), and attributing the detection to the correct protocol. Mean LOTO binary accuracy: 94.42%. Mean false positive rate: 0.91%.
- **MySQL 8.0 database:** Persistent storage of detection logs, user accounts, and session data in a normalized schema designed for efficient querying as log volume grows over extended deployments.
- **Security:** bcrypt password hashing, PHP session-based access control, HTTPS-enforced communication, and OWASP Top 10 compliant input sanitization across all user input fields and API endpoints.

- **Browser and OS compatibility:** Google Chrome, Mozilla Firefox, and Microsoft Edge on Windows 10 and Windows 11 (64-bit).

What Is Excluded

- Physical-layer and side-channel steganography, which operate below the network protocol level and cannot be detected through flow-level behavioral analysis.
- Application-layer media steganography in image, audio, video, or text files, which require entirely different detection approaches and carrier types.
- Real-time inline deployment engineering for packet interception and automated blocking. The system operates as an offline analytical detection tool.
- Active countermeasures, automated remediation, or network response actions. The system detects and presents evidence it does not block or quarantine.
- Linux and macOS deployment. Npcap requires Windows 10 or 11. Cross-platform extension is identified as a direction for future work.

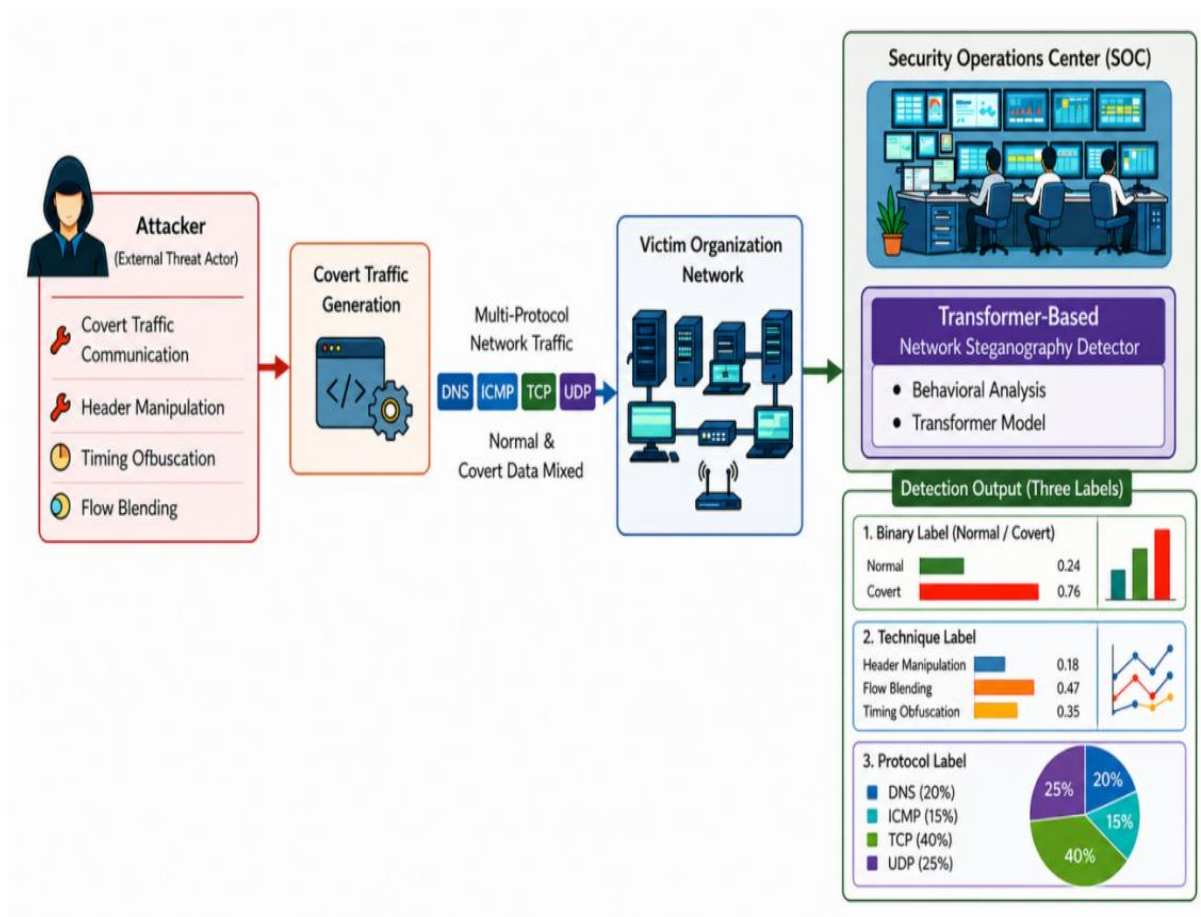


Figure 1.2. Illustration of covert data transmission methods across an IPv4 network

1.3. Purpose:

The purpose of StegFix is to bridge the gap between academically validated deep learning research on network steganography detection and a practically deployable tool that security analysts can use without specialist training. No existing publicly available tool integrates a validated model of this quality into a browser-accessible interface with live capture, real-time alerting, and forensic visualization. SOC analysts work with dashboards, not Python scripts they need clear visual indicators, exportable logs, and immediately actionable results. The secondary purpose is to demonstrate the feasibility of integrating deep learning inference into a web-based SOC workflow as a reference architecture for future researchers.

1.4. Objectives:

Detection Model Integration

- Integrate the pre-trained Feature Token Transformer (4 Attention Blocks, 8 attention heads, d_model=128) via a Python Flask microservice, achieving at least 90% binary detection accuracy across DNS, ICMP, TCP, and UDP.
- Validate correct technique classification across Header Manipulation, Timing Obfuscation, and Flow Blending with accurate protocol attribution per detected flow.

Web Application Development

- Implement secure user authentication with bcrypt hashing, PHP session-based access control, and HTTPS enforcement.
- Integrate live Npcap packet capture via Start/Stop dashboard controls, capturing source IP, destination IP, protocol, timestamp, and payload attributes without command-line access.
- Implement a real-time Dashboard with a live alerts panel, technique distribution chart, protocol traffic graph, and recent detections table.
- Implement Flow Detail Analysis, Technique Analysis, Protocol Analysis, and Detection Logs modules with CSV export.

- Ensure live graphs and alerts refresh at intervals not exceeding 5 seconds during active capture.

1.5 Tools and Technologies:

The system is implemented using a four-layer technology stack: frontend presentation, backend application, data persistence, and model inference.

Frontend: React.js with TypeScript serves as the primary frontend framework, providing component-based UI development with real-time state management and static type safety. HTML5 and CSS3 handle component structure and responsive layout. Chart.js and

Backend: PHP 8.2 implements the RESTful API backend, handling user authentication, session management, Npcap capture orchestration, feature vector routing to the inference endpoint, and all database operations. MySQL 8.0 provides persistent storage for the Users, Detection Logs, and Sessions tables in a third normal form normalized schema. XAMPP with Apache 2.4 hosts the PHP backend and MySQL database during development and controlled deployment.

Packet Capture and Detection: Npcap 1.78 provides low-level Windows packet capture integrated directly with the PHP backend, enabling Start/Stop capture controls on the dashboard without requiring command-line tools. The pre-trained Feature Token Transformer 4 Attention Blocks, 8-head multi-head self-attention, d_model=128, trained on 13,848 samples with 225 flow-level features is exposed as a /predict REST endpoint via Python Flask 2.3 running on localhost:5000. The PHP backend sends 225-feature vectors to this endpoint and receives JSON classification results. Wireshark 4.0.6 was used during development for traffic verification and PCAP replay.

Development Environment: Visual Studio Code served as the primary IDE. Git and GitHub provided version control and remote repository management. Node.js 18+ supports the React build toolchain via npm. The inference microservice runs on Python 3.9 with PyTorch 2.0 and Scikit-learn 1.3.2. The system requires Windows 10 or Windows 11 (64-bit) for Npcap compatibility and a minimum of 8 GB RAM to run all four system layers concurrently.

1.6 Chapter Summary:

This chapter established the foundation for StegFix. The introduction identified network steganography across DNS, ICMP, TCP, and UDP as a threat that signature-based systems, statistical detectors, classical ML, and deep learning detectors all fail to address reliably with LSTM accuracy documented falling from 99.4% to 76.9% under cross-technique evaluation. The proposed solution a web-based platform integrating the Feature Token Transformer achieving 94.42% mean LOTO accuracy and 0.91% mean false positive rate was introduced. The scope, purpose, objectives, and complete technology stack were defined. Chapter 2 presents the literature review and comparative analysis of existing systems.

Chapter 2: Literature Review & Existing Systems

2.1 Literature Review:

Network steganography detection is one of the most technically complex problems in modern cybersecurity. This chapter reviews the intellectual foundations of the proposed Transformer-based detection framework. The review is organized in four tiers: (1) theoretical foundations of network steganography; (2) traditional and classical machine learning detection approaches; (3) deep learning and Transformer architectures; and (4) cross-domain steganography research. The chapter identifies four research gaps and concludes with the problem statement, proposed solution, comparative analysis, and technical specification of the Feature Token Transformer model.

1. Theoretical Origins and Covert Channel Theory:

The theoretical basis of network steganography originates with Simmons [1], whose prisoners' problem defines the core detection challenge: two parties exchange hidden messages through a monitored channel while appearing to communicate legitimately. The guard representing any IDS, deep packet inspection engine, or security analyst must determine not whether message content is suspicious, but whether hidden communication is occurring at all, a fundamentally harder problem than signature matching.

Lampson [2] provided the complementary two-class covert channel taxonomy: storage channels convey information by modifying a shared storage location, and timing channels convey information through the relative timing of observable events. Both are exploitable at the network protocol level. This distinction maps directly to the three feature categories in the proposed dataset: Header Manipulation for storage channel anomalies, Timing Obfuscation for timing channel anomalies, and Flow Blending for hybrid manipulation.

Network Steganography Hiding Techniques and Tools: Wendzel et al. [3] provide the most complete protocol-agnostic taxonomy of network steganography hiding patterns. Storage-based techniques encode data in unused protocol fields DNS subdomain strings carry Base32 payloads, ICMP sequence numbers carry covert bits, and TCP initial sequence numbers embed hidden data. Timing-based techniques encode information in inter-packet delay patterns without any field modification, leaving no protocol violations. Hybrid techniques combine both approaches to increase covert bandwidth and defeat single-category detectors.

Operationally active tools include dns2tcp, dnscat2, and iodine for DNS; Ptunnel for ICMP; and Covert_TCP and HCovert for TCP. These tools are freely available and documented in real-world campaigns. Advanced persistent threat groups use them for data exfiltration, command-and-control communication, and long-term covert persistence [3].

Anti-Forensic Techniques: Real-world adversaries apply three primary anti-forensic transformations to defeat detection. Timing obfuscation adds Gaussian jitter to inter-packet delays, defeating statistical timing detectors. Header randomization varies field values within valid specification ranges, preventing tool fingerprinting. Flow blending adjusts packet size distributions and throughput statistics toward normal traffic baselines, defeating volume anomaly detectors. Systematic multi-protocol evaluation against all three transformations simultaneously is absent from existing detection research a primary gap this project addresses.

2. Traditional Detection Approaches:

Signature-based IDS systems such as Snort and Zeek match traffic against known attack signature databases. The limitation is structural a signature can only exist after a tool has been observed and documented. Any new or modified tool passes undetected. Rule-based protocol analysis flags atypical field usage but is defeated by techniques operating within valid protocol specification ranges.

Statistical approaches apply Chi-square and Kolmogorov-Smirnov tests to inter-packet delay distributions and entropy measurements to header fields. These achieve reasonable detection on clean traffic but fail against anti-forensic modifications: timing jitter defeats delay distribution tests, and header randomization produces entropy indistinguishable from legitimate traffic. Protocol behavior analysis requires separate calibrated models per protocol and fails against tools designed to mimic legitimate behavioral patterns [4].

3. Classical Machine Learning Approaches:

Support Vector Machines achieved 85%–92% accuracy in controlled settings using protocol-specific statistical features, and Random Forests demonstrated superior multi-class performance with basic feature importance interpretability [5]. Both share the critical feature engineering bottleneck: every input dimension must be manually created by a domain expert. Features designed for DNS tunneling do not describe ICMP covert traffic, requiring complete re-engineering per protocol. No classical ML approach has been evaluated against anti-forensically modified traffic in the reviewed literature.

Naive Bayes, k-Nearest Neighbors, and Decision Trees all perform below SVM and Random Forest and share the same bottleneck. The fundamental flaw of all classical ML approaches is that model discovery is bounded by the features designers chose to include the model cannot learn beyond what it was given.

4. Deep Learning Approaches:

CNNs achieve 87%–92% accuracy on network traffic classification but their bounded receptive field prevents capturing long-range temporal dependencies characteristic of timing-based covert channels [6]. LSTMs are designed for long-range dependency modeling and Cho and Lai reported 99.4% accuracy under standard supervised evaluation. However, accuracy collapsed to 76.9% on techniques absent from training data a 22.5 percentage point drop. This is operationally catastrophic: new steganographic tools are always absent from training datasets built before their release, making LSTM performance worst precisely when detection is most urgent.

Autoencoders detect anomalies without labeled steganographic examples but are limited because well-crafted covert traffic is designed to remain within normal statistical ranges. Graph Neural Networks achieve 80%–88% accuracy and outperform LSTM when inter-flow relational features are discriminating. Attention-enhanced LSTMs improve non-adjacent dependency modeling but retain the core sequential processing limitation of the underlying recurrent architecture.

5. Transformer Architectures in Network Security:

The Original Transformer Architecture: The Transformer introduced by Vaswani et al. replaced sequential recurrent processing with parallel multi-head self-attention, enabling direct computation of relationships between all positions in an input sequence simultaneously. The mechanism computes Query, Key, and Value projections: $\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$. Each position's output is a weighted combination of all other positions, with weights determined by dot-product similarity. Multi-head attention executes h independent attention functions in parallel, allowing different heads to specialize in header entropy relationships, timing dependencies, and flow volume co-variations simultaneously. For steganography detection, only the encoder component is used, producing a contextually enriched representation across all input features for classification.

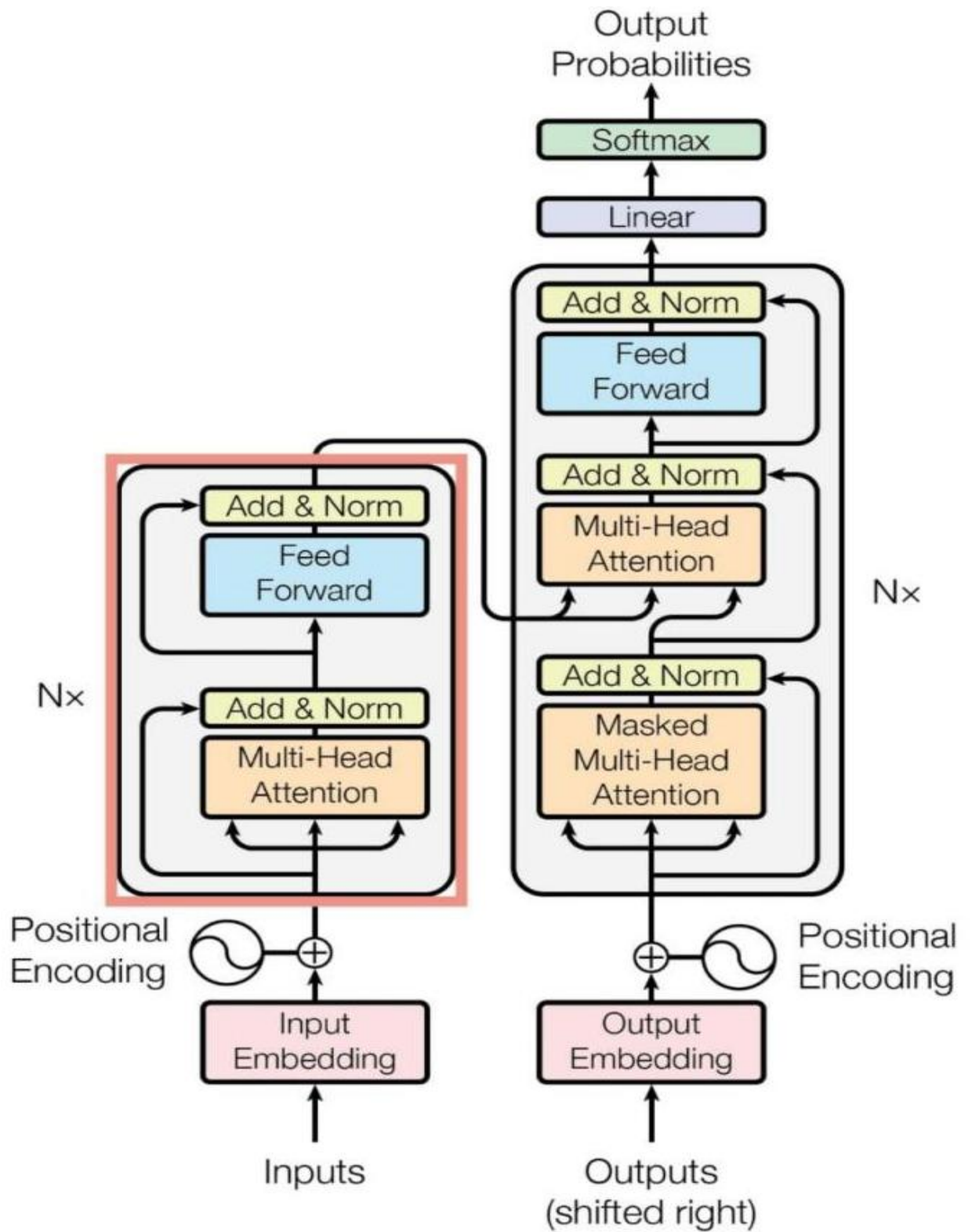


Figure 2.1. Standard Transformer encoder-decoder architecture

Transformers in Network Security and Prior DNS Work: Transformer applications in network security expanded significantly between 2020 and 2025, covering intrusion detection, malware classification, and log anomaly detection. The most directly relevant prior work is Sun et al. , who applied a DistilBERT-based Transformer to DNS steganography detection and reported 99.9% accuracy under supervised evaluation, outperforming LSTM's 76.9% few-shot performance [6][7]. However, Sun et al. [8] has three critical limitations: the framework covers DNS only; no anti-forensic modification evaluation is conducted.

Why Transformers Suit Network Steganography Detection: The same computation processes DNS, ICMP, TCP, and UDP vectors without protocol-specific modifications, enabling a single unified architecture for multi-protocol detection. Attention weight matrices provide forensic interpretability: for each flagged flow, the 225×225 heatmap identifies which feature category Header Manipulation, Timing Obfuscation, or Flow Blending.

Cross-Domain Steganography Insights: Research across image, audio, video, and linguistic steganography validates two principles directly applicable to network steganography detection. First, neural models learn the statistical structure of carrier media without explicit feature specification image steganalysis networks detect subtle LSB modifications and linguistic models identify semantically inconsistent embedded words, confirming that representation learning generalizes across steganography carrier types. Second, detectors evaluated only on clean default-configuration traffic will be defeated by tools optimized against them.

2.2. Problem Statement:

All existing detection approaches fail across five compounding dimensions: (1) signature-based systems cannot detect protocol-compliant covert traffic with no known signature; (2) classical ML feature engineering is protocol-specific and does not generalize across protocols or techniques; (3) deep learning detectors overfit to training distributions and collapse on unseen techniques LSTM accuracy fell from 99.4% to 76.9% under cross-technique evaluation ; (4) no existing system has been evaluated against adversarial anti-forensic modifications; (5) no existing multi-protocol system provides interpretable detection reasoning usable in forensic investigations. No single approach delivers multi-

protocol accuracy, behavioral generalization, anti-forensic robustness, and forensic interpretability within one evaluated framework.

2.3. Proposed Solution:

The proposed solution is a three-layer architecture: a Feature Token Transformer detection model, a PHP RESTful API backend, and a React.js analyst dashboard. The Feature Token Transformer treats each of 225 flow-level behavioral features as an independent input token, enabling multi-head self-attention to compute relationships across all features simultaneously. In a single forward pass it performs binary covert/normal detection, steganographic technique identification across three categories, and protocol attribution across DNS, ICMP, TCP, and UDP.

All traffic identifiers source IP, destination IP, ports, timestamps, and tool labels are removed before training, forcing the model to learn exclusively from behavioral patterns rather than tool-specific signatures. The model was trained on 13,848 samples across all four protocols and evaluated using the Leave-One-Technique-Out protocol. The web platform provides live Npcap packet capture, real-time alerts, per-protocol flow analysis, technique visualization, and CSV log export through a browser-accessible dashboard.

2.4. Comparative Analysis between Existing and Proposed Solution

Table 2.1. Comparative Analysis Existing Detection Approaches vs. Proposed Solution

Criterion	Signature-Based IDS	Statistical Methods	LSTM / CNN Detectors	Proposed Solution
Detection of Unknown Techniques	No reactive only	No threshold-based	Partial collapses on unseen (LSTM: 76.9%)	Yes 94.42% LOTO mean accuracy
Multi-Protocol Coverage	Limited to known signatures	Protocol-specific only	Single protocol in reviewed work	DNS, ICMP, TCP, UDP unified
Behavioral Generalization	None	None	No identifier-retaining training	Yes, identifier-removal enforced before training
Anti-Forensic Robustness Evaluation	None	None	Not evaluated in any reviewed work	Evaluated across 3 evasion categories
Real-Time Web Interface	No	No	No	Yes, React.js dashboard
Live Packet Capture	External tool required	External tool required	No	Integrated Npcap capture
False Positive Rate	High	Medium–High	Low on known techniques only	0.91% mean across all LOTO splits
Forensic Interpretability	No	Limited	Binary flag only	Attention weight feature contribution indicators

Export / Audit Logging	Limited	None	None	CSV export with full detection metadata
Deployment Accessibility	Specialist config required	CLI / scripting required	Research prototype only	Browser-accessible, no specialist training required

2.5. Deep Learning Technique: The Feature Token Transformer

Architecture: Input is a tensor of shape (batch, 225, 1). Each scalar feature is projected to a 128-dimensional token embedding via a linear layer. Four Attention Blocks are applied in sequence, each performing 8-head multi-head self-attention over all 225 tokens (16 dimensions per head), followed by a position-wise feed-forward network (128→256→128, ReLU), with residual connections, layer normalization, and dropout at 0.3 throughout.

Dataset Generation: The dataset contains 13,848 samples split 80/20 into 11,078 training and 2,770 test samples using stratified splitting. Each sample contains 225 flow-level behavioral features distributed across DNS (66), ICMP (97), TCP (41), and UDP (26) protocol-specific subsets, organized into three categories: 107 Header Manipulation features, 50 Timing Obfuscation features, and 73 Flow Blending features.

DNS data was sourced from the Kaggle DNS Tunnelling dataset, containing pre-extracted flow features from 10 covert tools cobalt_strike, dns2tcp, dnscat2, DNSExfiltrator, DNSlivery, iodine, OzymanDNS, reverse_dns_shell, tcp-over-dns, and tuns plus benign samples.

ICMP, TCP, and UDP data was self-generated in an isolated virtual machine laboratory (Kali Linux attacker VM, Ubuntu 20.04 victim VM, host-only network, no external connectivity) using Ptunnel, Covert_TCP, Scapy, and Python socket libraries.

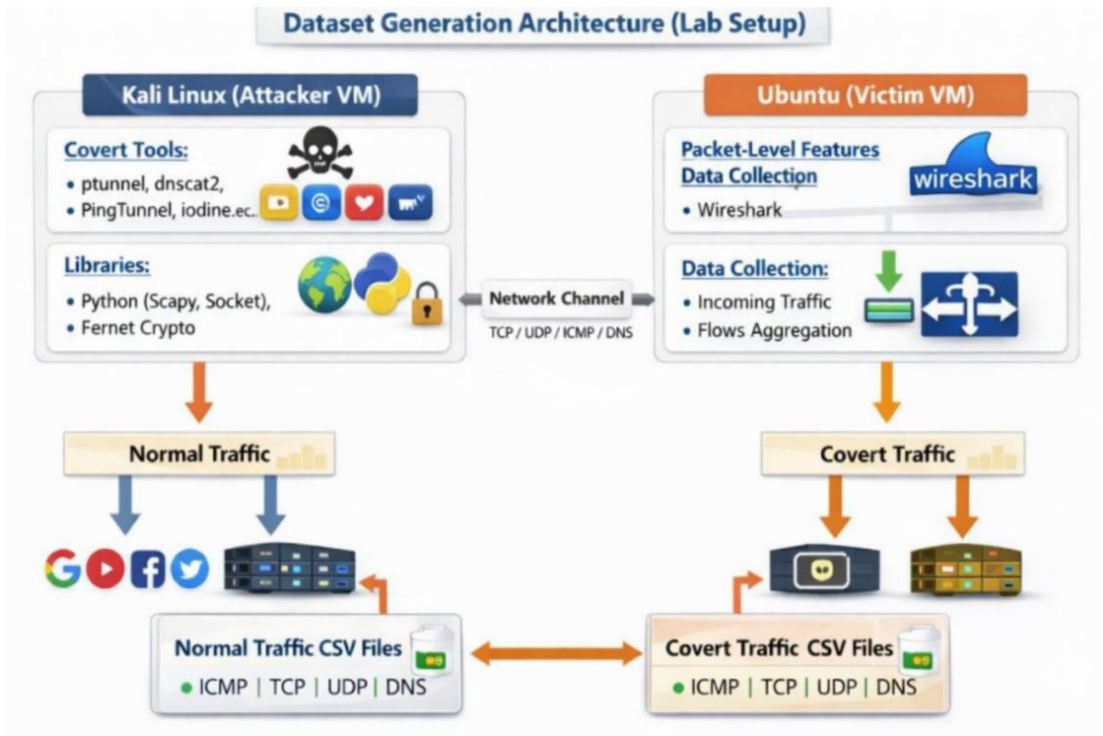


Figure 2.2. Dataset Generation Architecture Lab Setup

Dataset Creation and Preprocessing Pipeline: The dataset was constructed by collecting network traffic from multiple protocols including DNS Tunneling, ICMP, TCP, and UDP. Each protocol represents different types of network behavior including both normal and covert communication - DNS Tunneling: Used to simulate covert channels using DNS queries- ICMP: Represents ping-based communication- TCP: Includes standard connection-based traffic. UDP: Represents connectionless communication pattern.

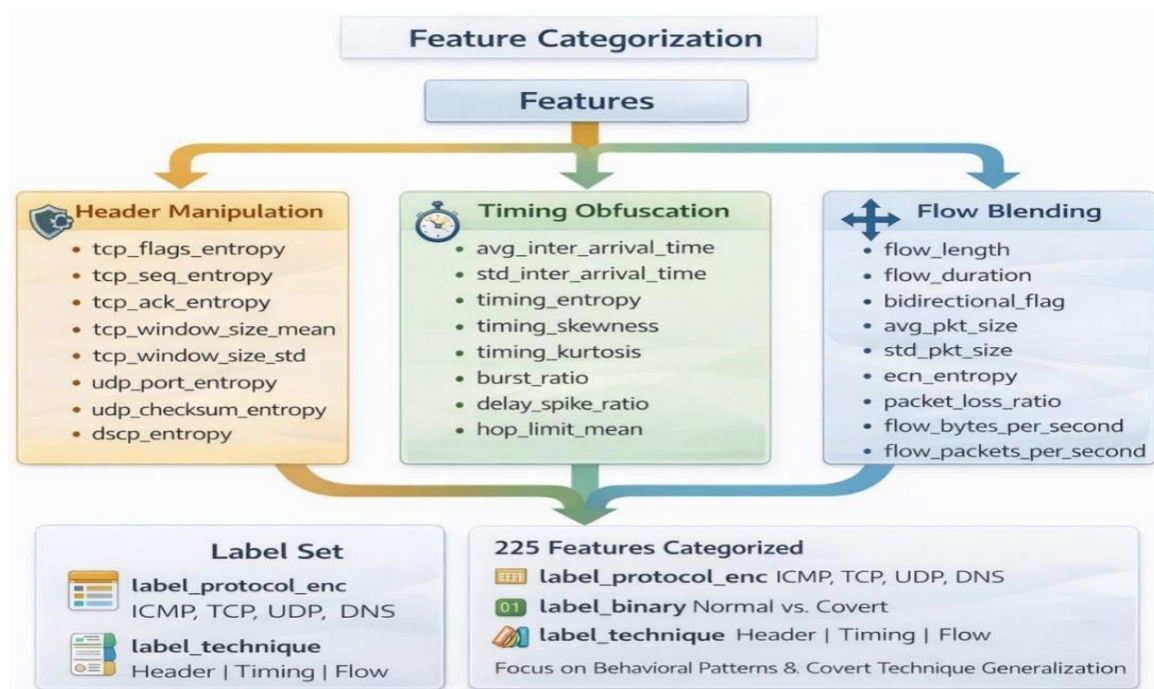


Figure 2.3. Feature Categorization Three-Category Framework

Packet to flow level Features conversion: Converts packet level to flow level features through self-generated python scripts.

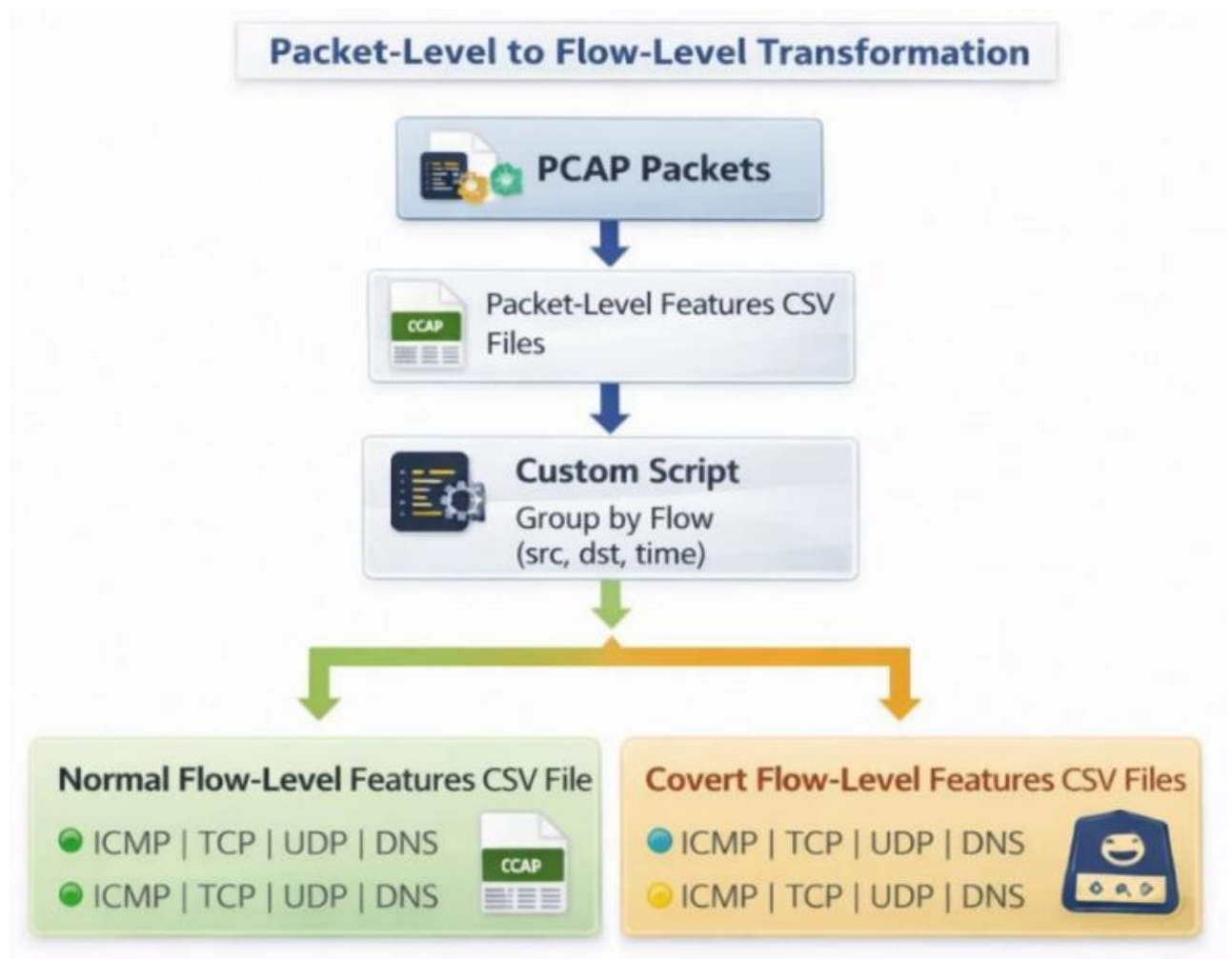


Figure 2.4. Packet-level to flow-level transformation pipeline

Raw Dataset Construction (FINAL_MASTER_RAW_DATASET)

All protocol-specific datasets were merged into a single master dataset called FINAL_MASTER_RAW_DATASET.

Steps:

1. Load individual protocol datasets (DNS, ICMP, TCP, UDP)
2. Standardize column names across datasets
3. Add labels (Normal / Covert)
4. Concatenate all datasets into one unified dataset
5. Shuffle dataset to remove ordering bias

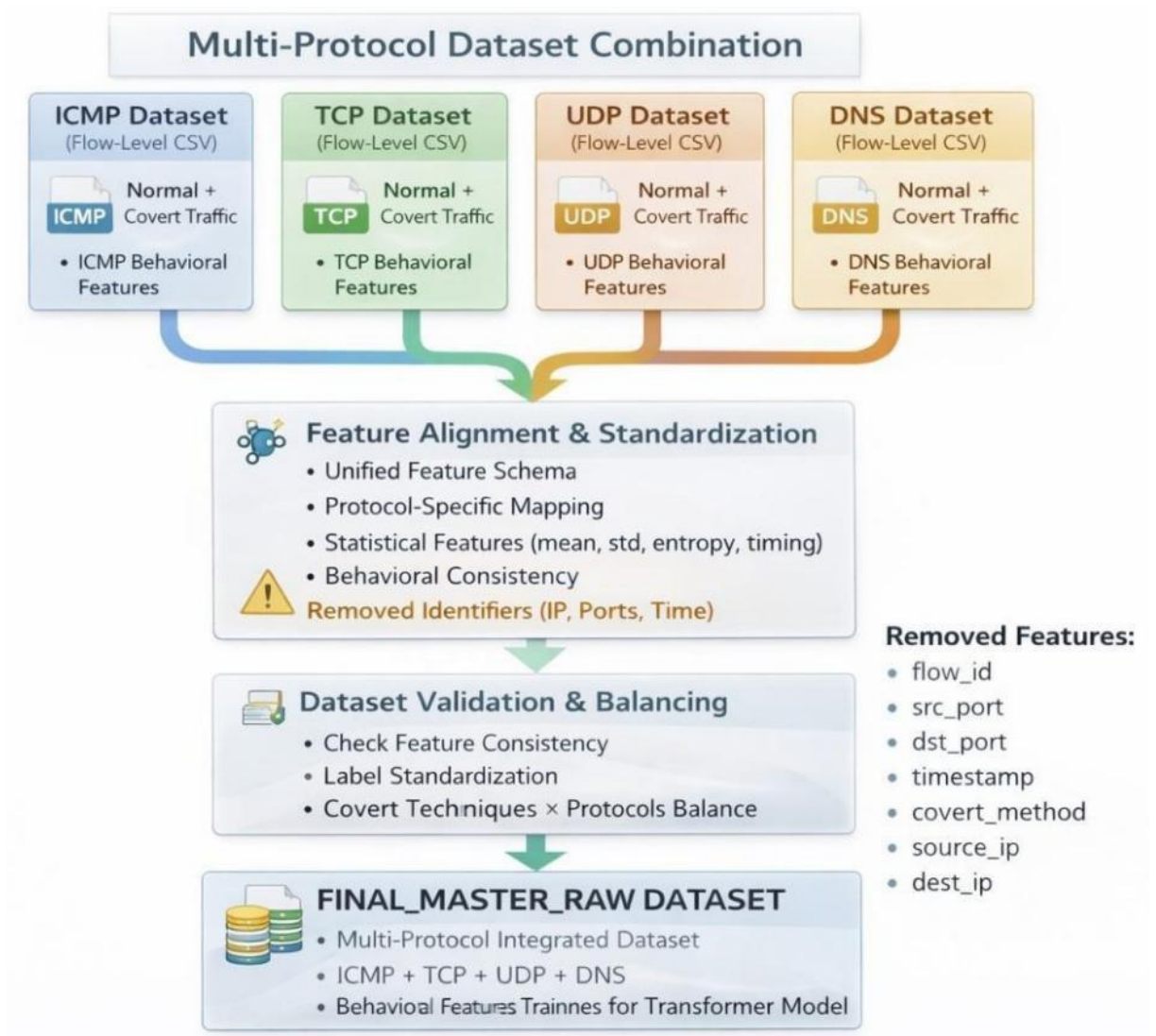


Figure 2.5. Multi-Protocol Dataset Combination and Preprocessing Pipeline

Data Cleaning: Identifier removal: All source IP, destination IP, port number, and timestamp fields removed from all protocol datasets before training to enforce behavioral generalization. Removed duplicate records. Handled missing/null value.

Labeling Strategy: Each data instance was labeled as:

label_binary: 0 → Normal Traffic, 1 → Covert / Steganographic Traffic

label_technique: 0 → Normal, 1 → header manipulation, 2 → Timing obfuscation,

3 → Flow blending

label_protocol: 0 → DNS, 1 → ICMP, 2 → TCP, 3 → UDP

Preprocessing Pipeline The preprocessing stage converts raw data into model-ready format:

Step 1: Load FINAL_MASTER_RAW_DATASET

Step 2: Clean dataset

Step 3: Extract features

Step 4: Encode labels

Step 5: Normalize features

Step 6: Split dataset

Step 7: Convert to tensors Output: **preprocessed_master_dataset.csv**

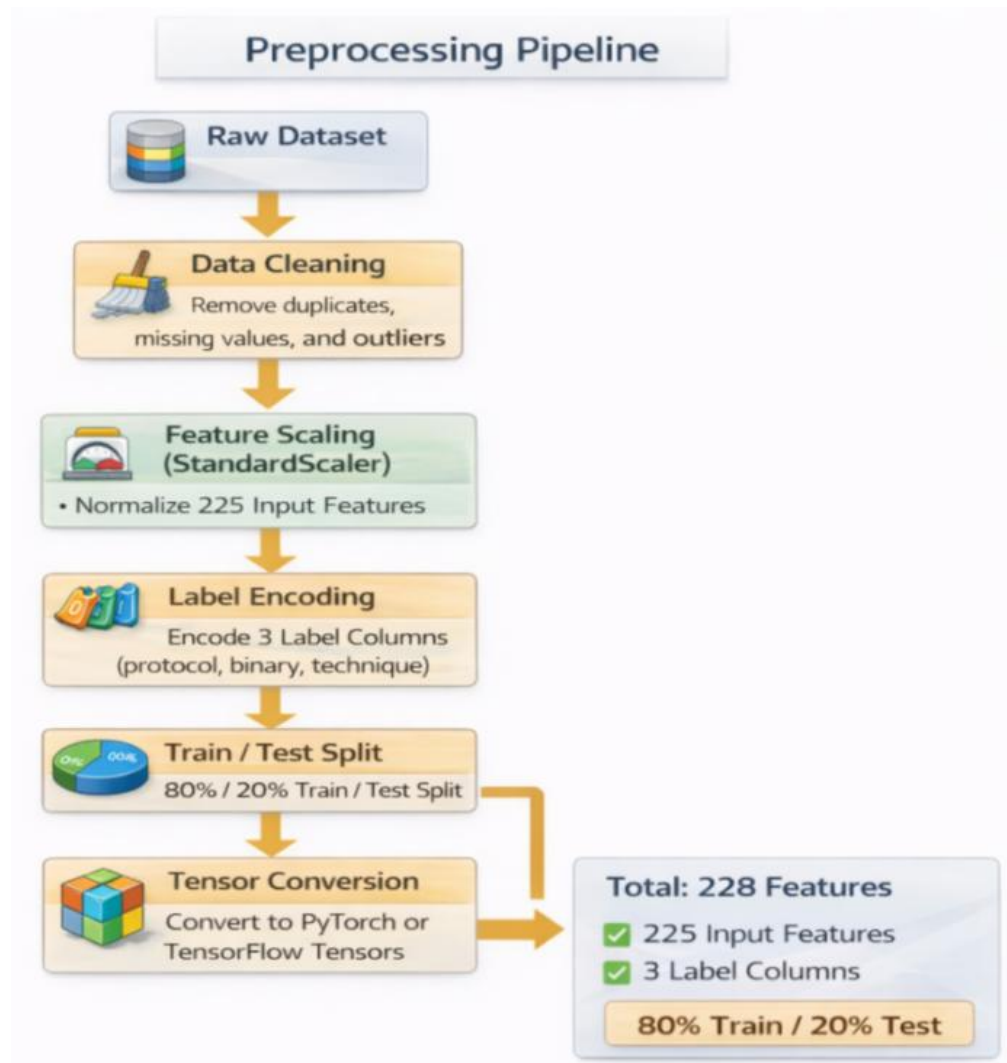


Figure 2.6. Preprocessing pipeline of our proposed model

Feature Scaling and Normalization All numerical features were scaled using Standard Scaler. To ensure equal contribution of all features during training.

Behavioral Learning Strategy

Instead of signature-based detection, the model learns how traffic behaves. This allows detection of unknown attacks. The model is trained on behavioral patterns such as:

- ✓ Timing irregularities
- ✓ Flow distribution changes
- ✓ Packet size variations

2.6. Chapter Summary

This chapter traced the intellectual evolution of network steganography detection from Simmons' covert channel theory and Lampson's storage/timing channel taxonomy, through traditional signature-based and statistical approaches, classical ML methods, and deep learning architectures, to the identified failure point documented by Cho and Lai LSTM accuracy collapsing from 99.4% to 76.9% under cross-technique evaluation. The Transformer architecture's global self-attention, protocol-agnostic processing, and forensic interpretability through attention visualization were established as the technically superior approach for multi-protocol detection. Cross-domain steganography research from image, audio, video, and linguistic domains validated the neural representation learning approach and the adversarial evaluation imperative. Four research gaps were identified: the absence of multi-protocol Transformer detection, behavioral generalization through identifier removal, systematic anti-forensic robustness evaluation, and multi-protocol forensic interpretability. The proposed Feature Token Transformer addresses all four simultaneously, achieving 94.42% mean LOTO binary accuracy and a 0.91% mean false positive rate.

Chapter 3: Software Requirements Specification

3.1 Functional Requirements

The system is composed of seven functional modules:

User Authentication: The system shall allow analysts to register with a unique username, email, and password. Login validates credentials against bcrypt hashed database values and creates a session token. Users can logout at any time, invalidating the session token.

Traffic Capture: The system shall integrate Npcap via Start/Stop controls on the Dashboard to initiate and terminate live packet capture. Captured packets display source IP, destination IP, protocol, timestamp, and payload attributes in real time. A live alerts panel updates with detected covert host IPs, and a recent detections table shows Timestamp, Source IP, Destination IP, Protocol, Technique, Risk Level, and Status.

Dashboard Analytics: The Dashboard shall display a doughnut/pie chart showing the proportional distribution of the three technique categories Header Manipulation, Timing Obfuscation, and Flow Blending and color-coded risk classification labels (Low, Medium, High) for recent detection events.

Flow Detail Analysis: A tabbed page shall provide per-protocol flow inspection for TCP, UDP, ICMP, and DNS. Each tab displays the Feature Token Transformer input features with structured field labels and a horizontal bar chart showing Normal versus Covert traffic percentage for that protocol.

Technique Analysis: Four protocol-specific pages shall each display: a bar chart of Normal versus Covert traffic distribution; a pie chart for Flow Blending proportions; and a line chart showing the temporal pattern of Timing Obfuscation events across the session timeline.

Protocol Analysis: The system shall display total packet counts and detected covert flow counts for all four protocols simultaneously, with a real-time bar graph refreshing at intervals not exceeding five seconds during active capture.

Logs and Export: All detection events shall be persistently stored in MySQL with fields: Timestamp, Source IP, Destination IP, Protocol, Technique, Risk Level, and Status. Authenticated users can export all visible log records as a CSV file via an Export button.

3.2. Non-Functional Requirements

Performance:

- End-to-end classification latency (capture → inference → dashboard) shall not exceed seconds per flow.
- All pages shall load within 3 seconds of an authenticated request.
- Live graphs and alerts shall refresh at intervals not exceeding 5 seconds during active capture.

Security:

- All passwords shall be stored using bcrypt. Plain-text storage is prohibited.
- PHP session-based access control shall protect all pages, API endpoints, and export functions.
- All frontend-to-backend communication shall use HTTPS with TLS 1.2 or higher.

Usability:

- The interface shall target a minimum SUS score of 70, enabling analysts to start captures, view results, and export logs without documentation.
- Clear visual feedback shall indicate all key states: active capture, detection alerts, errors and successful export.

Reliability:

- The system shall maintain 99% uptime during active capture sessions with no crashes or data loss.
- Network interface errors shall display informative messages without crashing the application.
- All detection logs shall survive a complete system restart with zero data loss.

Maintainability and Scalability:

- Frontend: React.js with TypeScript for component modularity and type safety.
- Backend: PHP RESTful API with clean separation of concerns.
- Model: Decoupled via Flask inference microservice model updates require no frontend or backend changes.
- Database: MySQL schema normalized to at least third normal form with performance indexes supporting growing log volumes.

Compatibility:

- The web application shall function fully in Chrome, Firefox, and Edge.
- Live capture shall be compatible with Windows 10 and Windows 11 (64-bit).

3.3 Use Case Diagram:

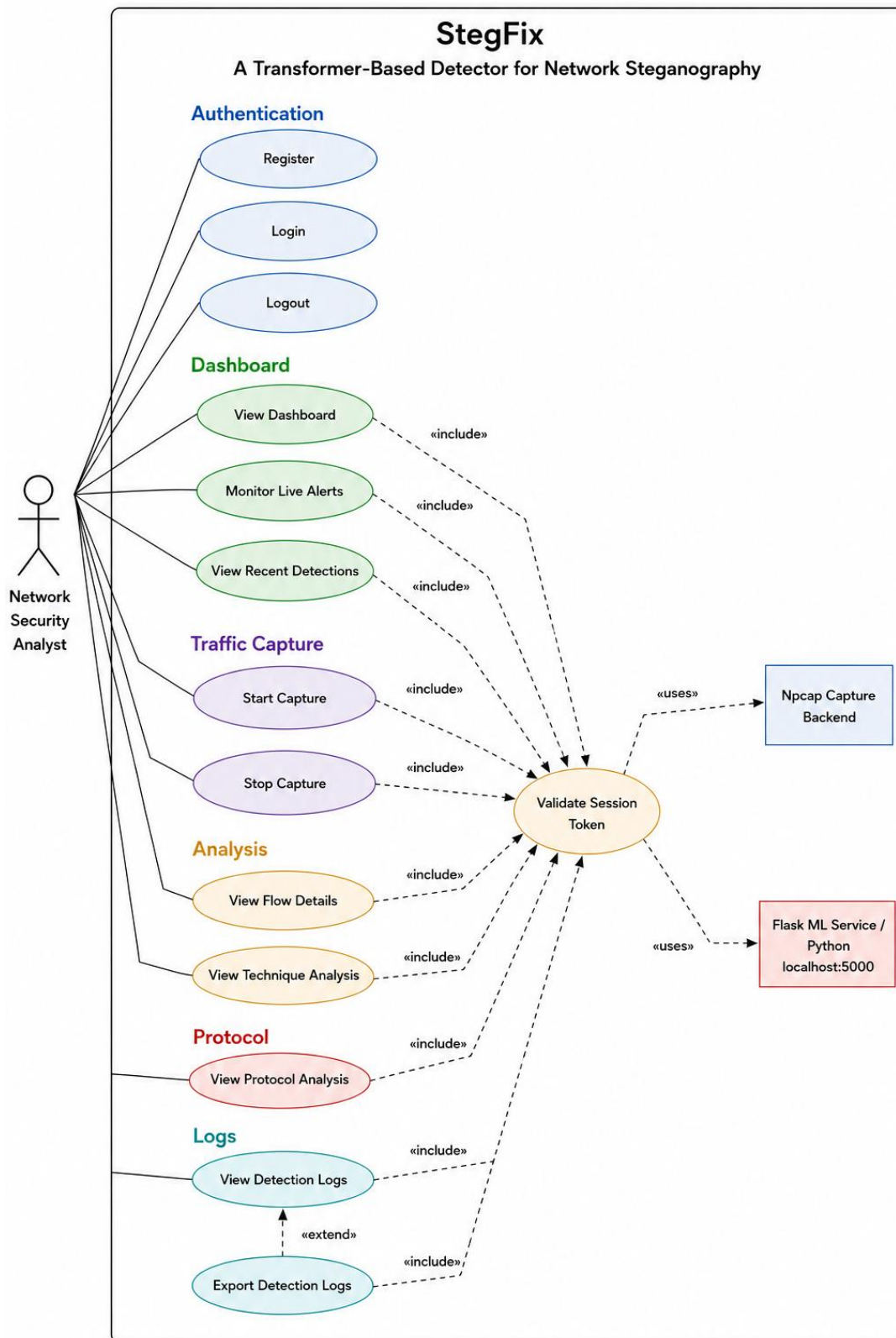


Figure 3.1. Use Case Diagram of Stegfix

3.4 Methodology and Software Lifecycle:

Selected Methodology: Agile with Iterative Development

The development of this system followed an Agile Software Development Methodology with iterative sprint-based delivery. Agile was selected as the governing methodology rather than Waterfall or Spiral models because of the research-oriented and experimental nature of this Final Year Project.

Software Development Lifecycle (SDLC) Phases

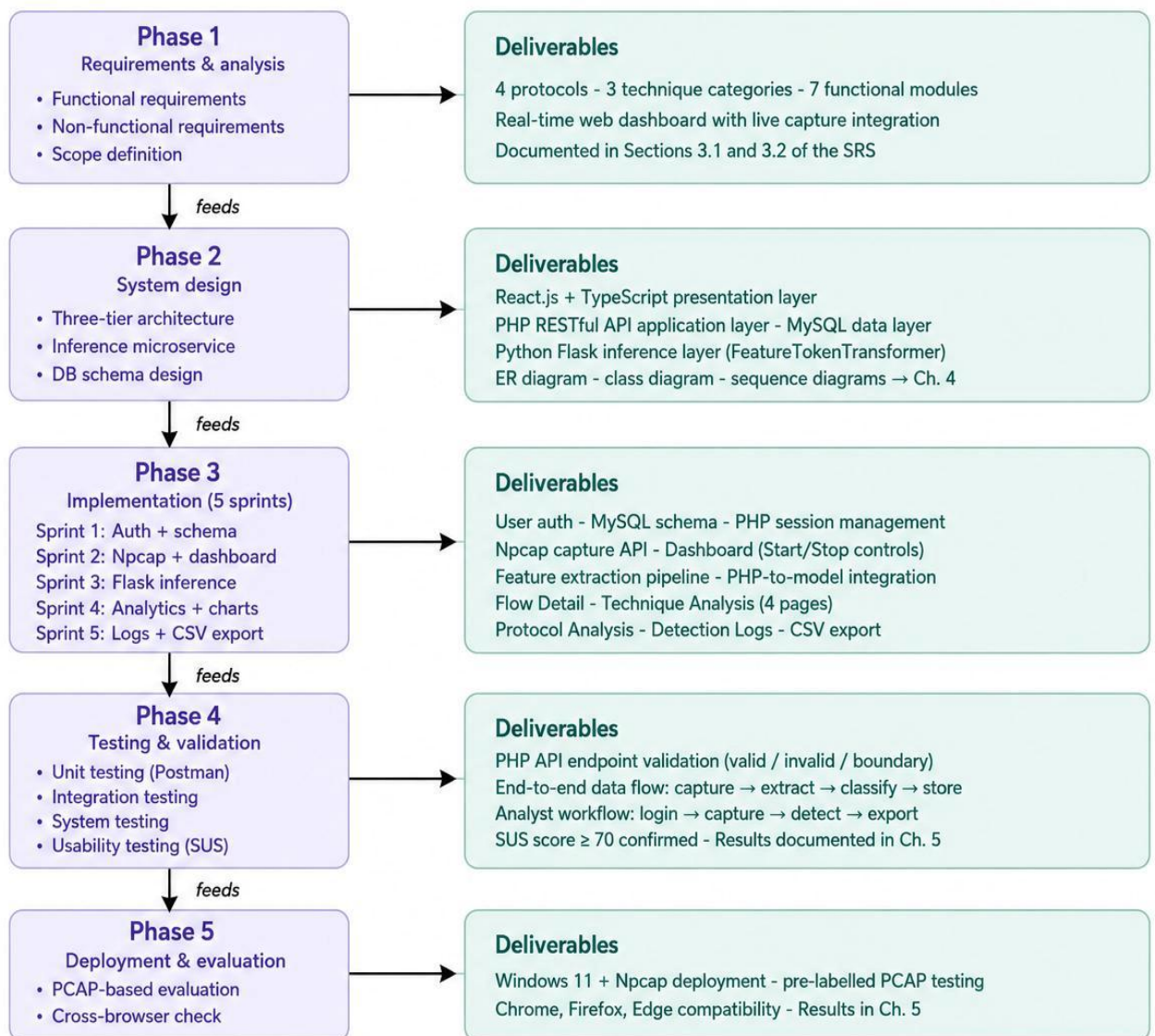


Figure 3.2. Software Development Lifecycle (SDLC) Phases of Stegfix

3.5 Chapter Summary

This chapter presented the complete Software Requirements Specification for the network steganography detection platform. Twenty-two functional requirements were defined across seven modules: User Authentication, Traffic Capture, Dashboard Analytics, Flow Detail Analysis, Technique Analysis, Protocol Analysis, and Logs and Export. Twenty non-functional requirements were defined across six quality dimensions: performance, security, usability, reliability, maintainability and scalability, and compatibility.

A use case diagram illustrated the complete set of analyst interactions across all system modules. The Agile Software Development Methodology was adopted and justified as the most appropriate approach for this research-intensive dual-track project, with five SDLC phases defined Requirements, Design, Implementation, Testing, and Deployment each with clear deliverables linking to subsequent chapters.

Chapter 4: Design and Architecture

4.1 Entity Relationship Diagram (ERD):

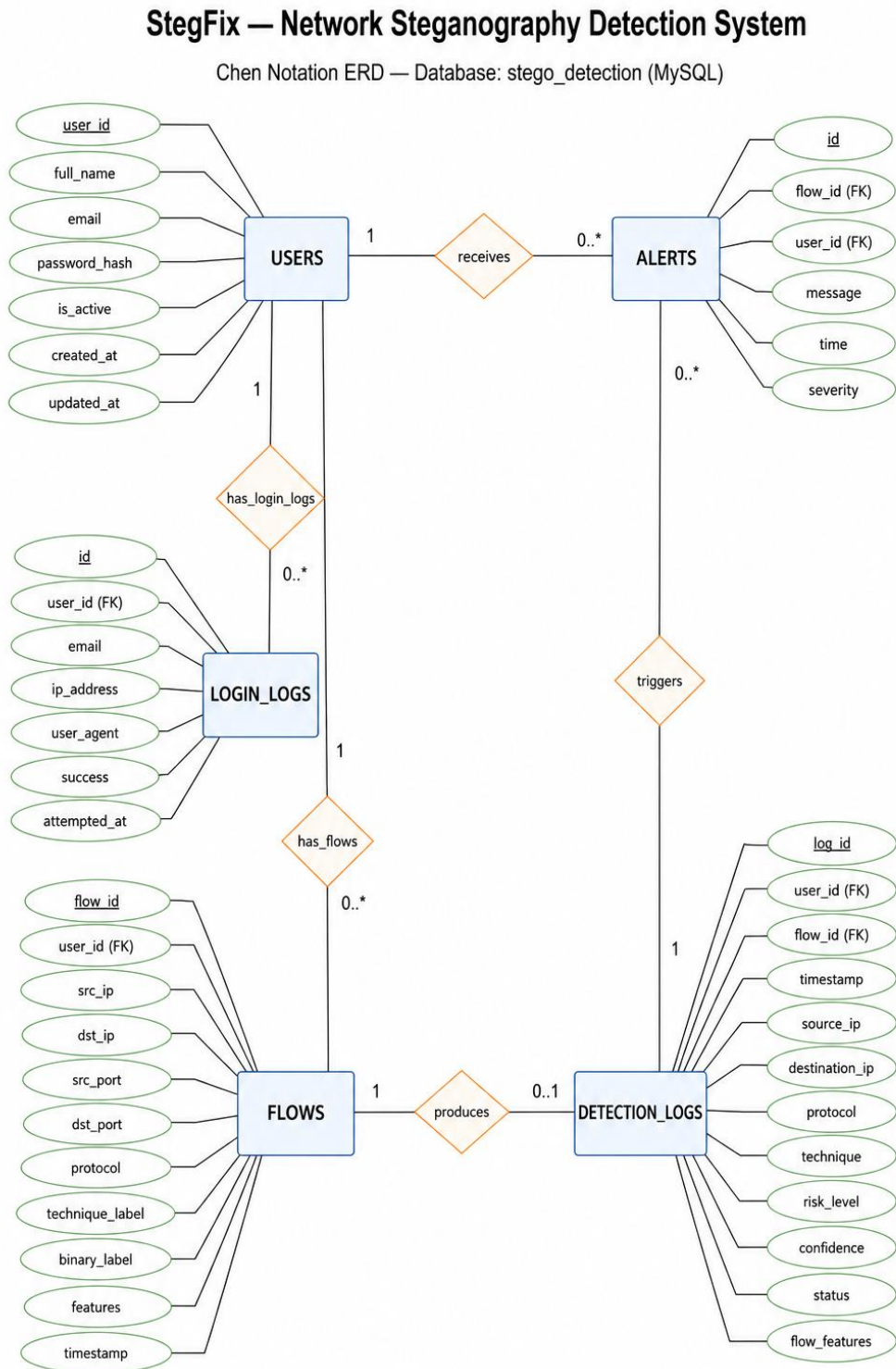


Figure 4.1. Entity Relationship Diagram of Stegfix

4.2 Sequence Diagrams:

StegFix — User Authentication Sequence Diagram

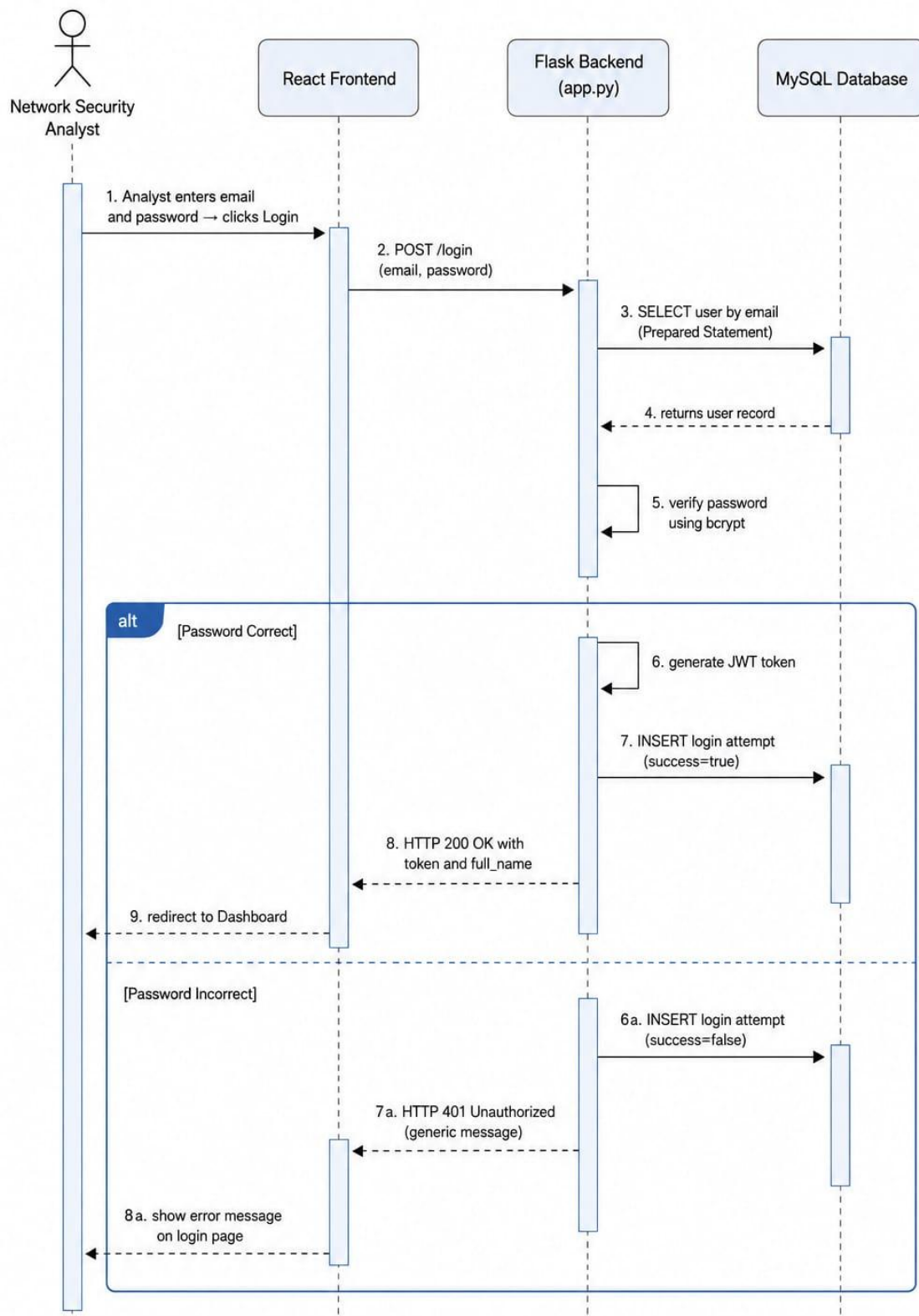


Figure 4.2. Sequence Diagram - User Authentication Flow

4.2.2 Live Capture and Detection Sequence

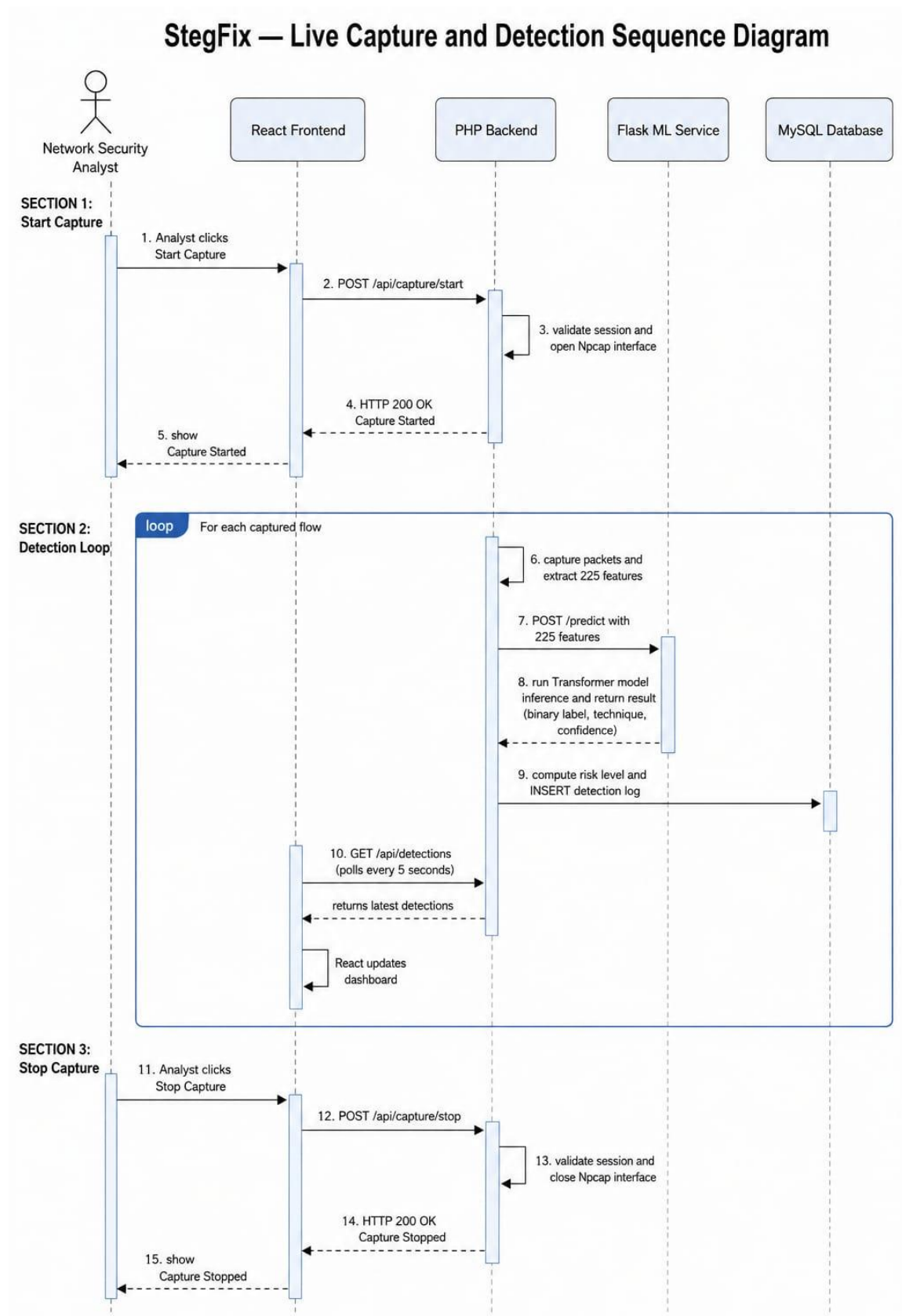


Figure 4.3. Sequence Diagram Live Capture and Detection Flow

4.2.3 CSV Log Export Sequence

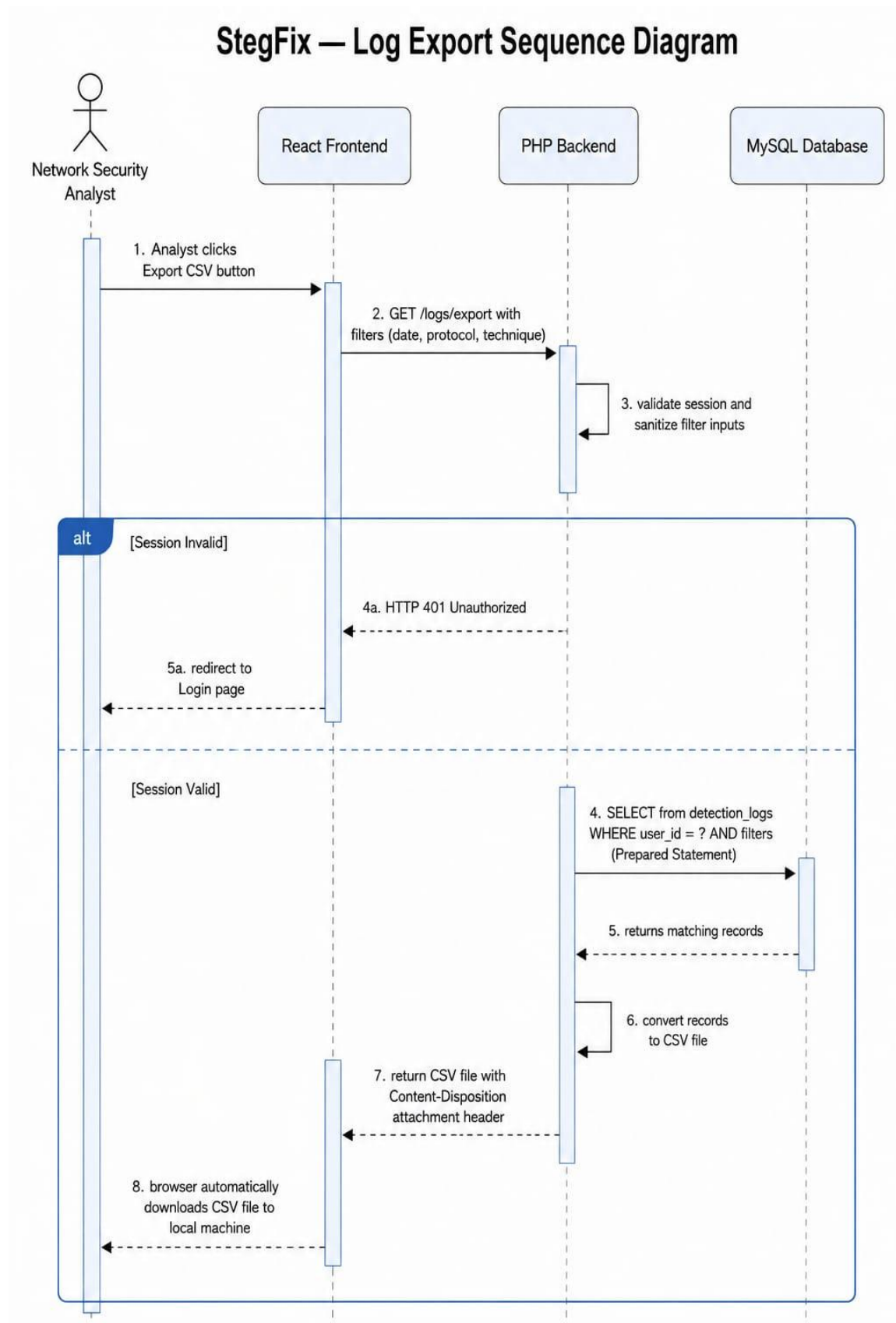


Figure 4.4. Sequence Diagram - CSV Log Export Flow

4.3 Architecture Design Diagram:



Figure 4.5. System Architecture Design Diagram of Stegfix

4.4 Class Diagram:

StegFix — Network Steganography Detection System — Class Diagram

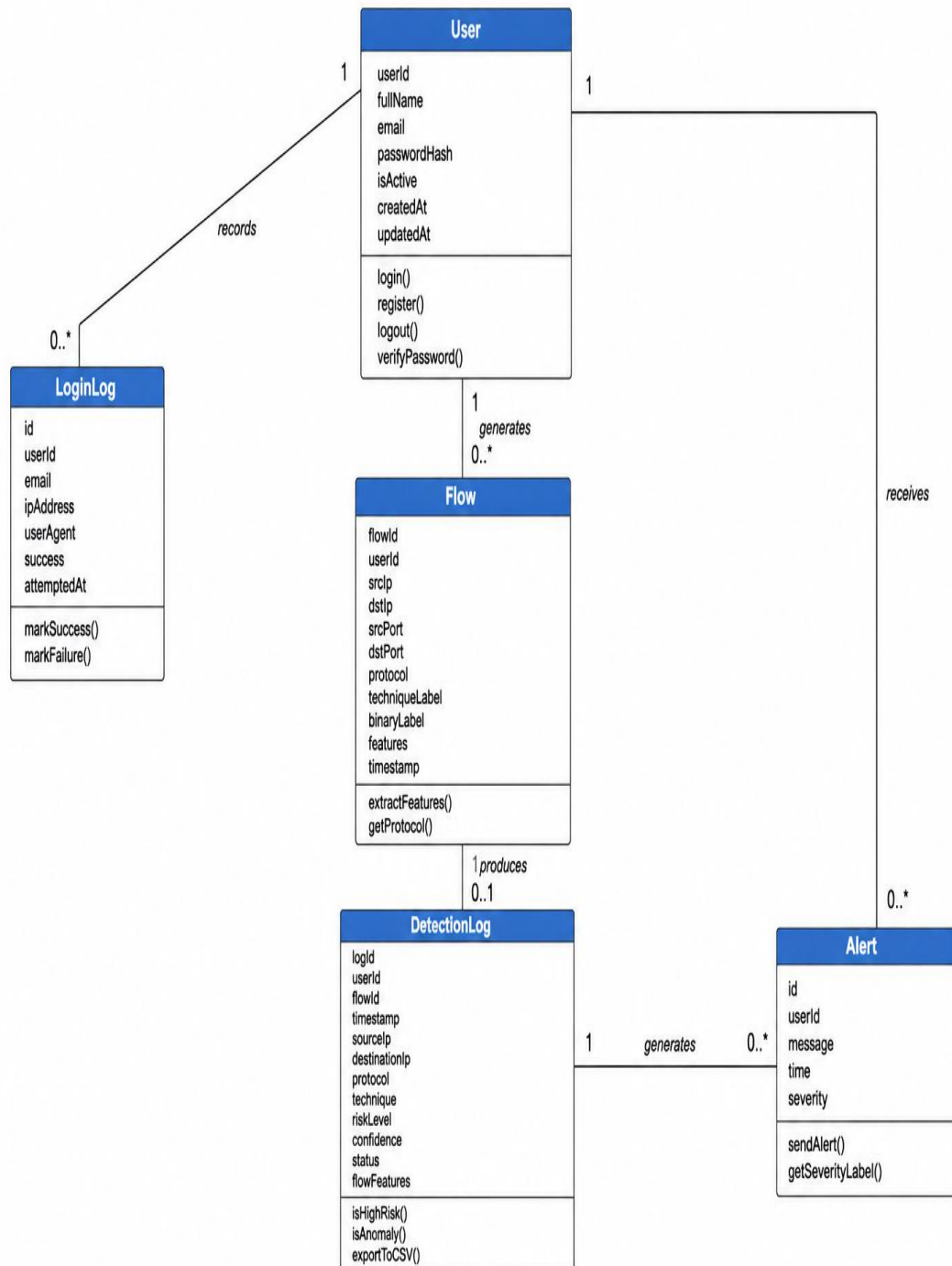


Figure 4.6. Class Diagram of Stegfix

4.5 Database Design:

The MySQL database schema is designed to third normal form. All three tables are presented below with their complete column specifications. Foreign key constraints enforce referential integrity between the tables.

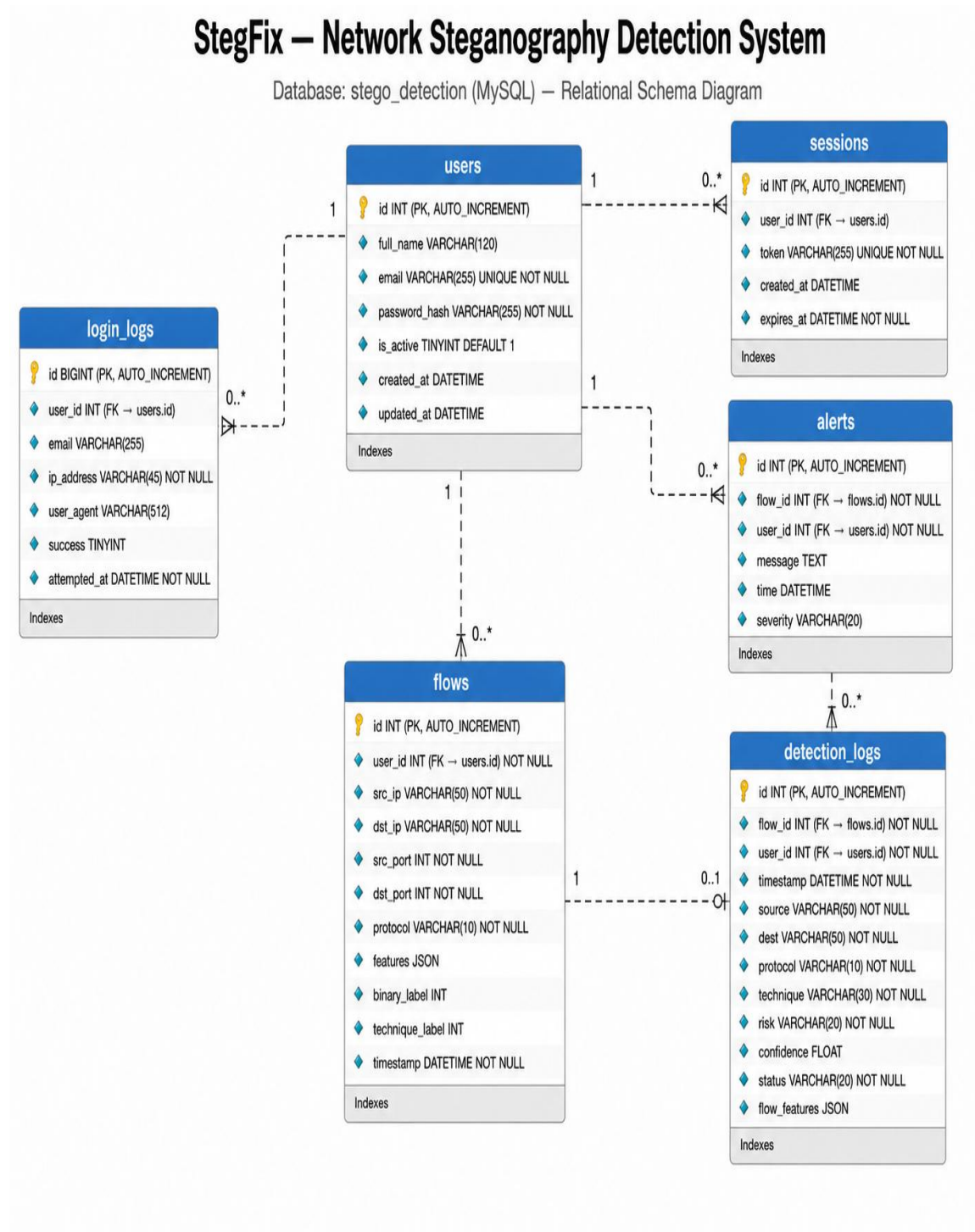


Figure 4.7. Database Design of Stegfix

4.6 Interface Design:

Registration and Login Interface:

The figure displays two screenshots of the StegFix web interface, which features a dark blue background with a network-like pattern of dots and lines.

Top Screenshot (Registration Interface):

- Header:** StegFix logo with a shield icon and the text "Create Your Account". A "Back to Login" button is in the top right corner.
- Form Fields:**
 - Full Name:** Input field with a person icon and placeholder text "Enter your full name".
 - Email Address:** Input field with an envelope icon and placeholder text "Enter your email".
 - Password:** Input field with a lock icon and placeholder text "Create a password".
 - Confirm Password:** Input field with a lock icon and placeholder text "Re-enter your password".
- Buttons:** A large blue "Create Account" button.
- Footer:** Text "Already have an account? [Sign in](#)" and "Authorized personnel only".
- Windows Notification:** "Activate Windows. Go to Settings to activate Windows."

Bottom Screenshot (Login Interface):

- Header:** StegFix logo with a shield icon and the text "Secure Access Portal". A "Sign Up" button is in the top right corner.
- Form Fields:**
 - Email Address:** Input field with an envelope icon and placeholder text "laiba@gmail.com".
 - Password:** Input field with a lock icon and placeholder text "*****".
- Buttons:** A large blue "Login" button.
- Form Elements:** A "Remember me" checkbox and a "Forgot password?" link.
- Footer:** Text "Don't have an account? [Create one](#)" and "Authorized personnel only".
- Windows Notification:** "Activate Windows. Go to Settings to activate Windows."
- Page Footer:** "Academic Research System v2.0" at the bottom center.

Figure 4.8: Login Interface Design of Stegfix

Dashboard Interface:

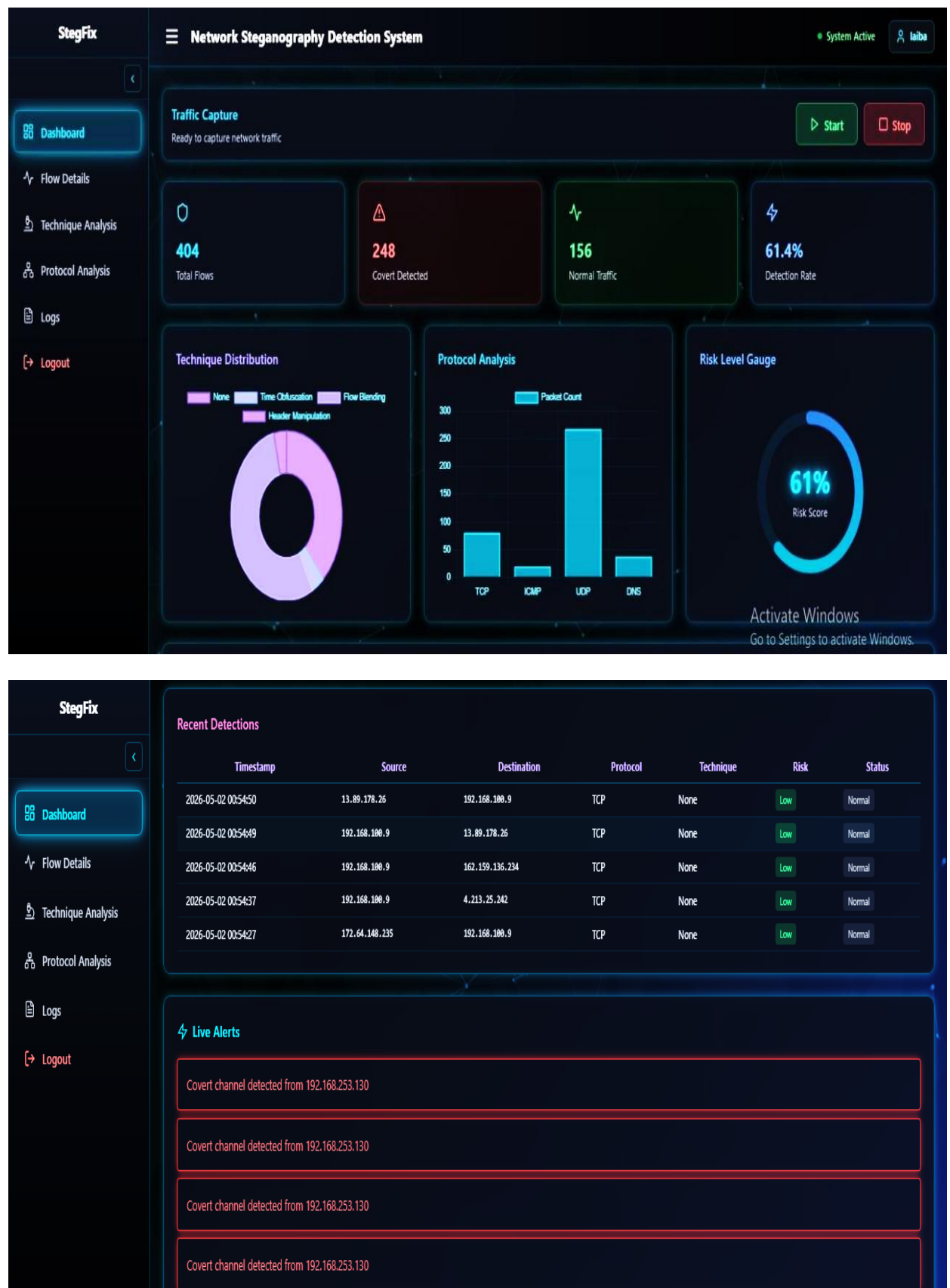


Figure 4.9. Dashboard Interface Design of Stegfix

Flow Detail Interface:

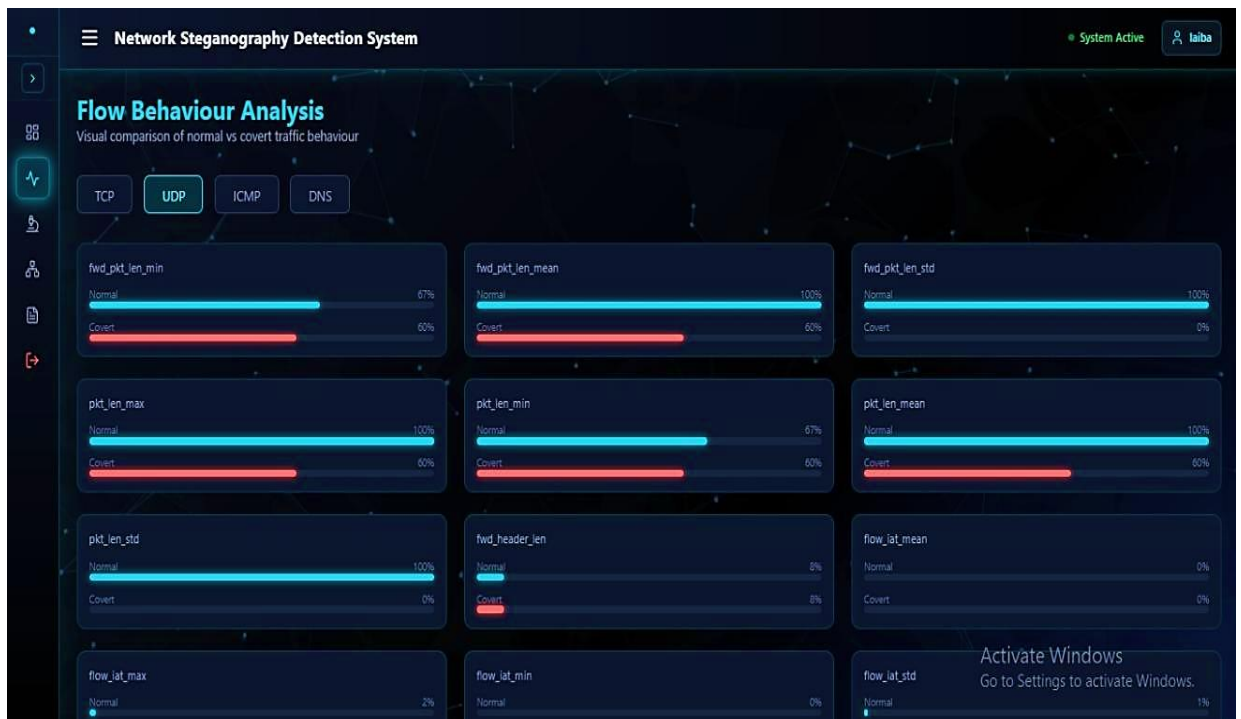
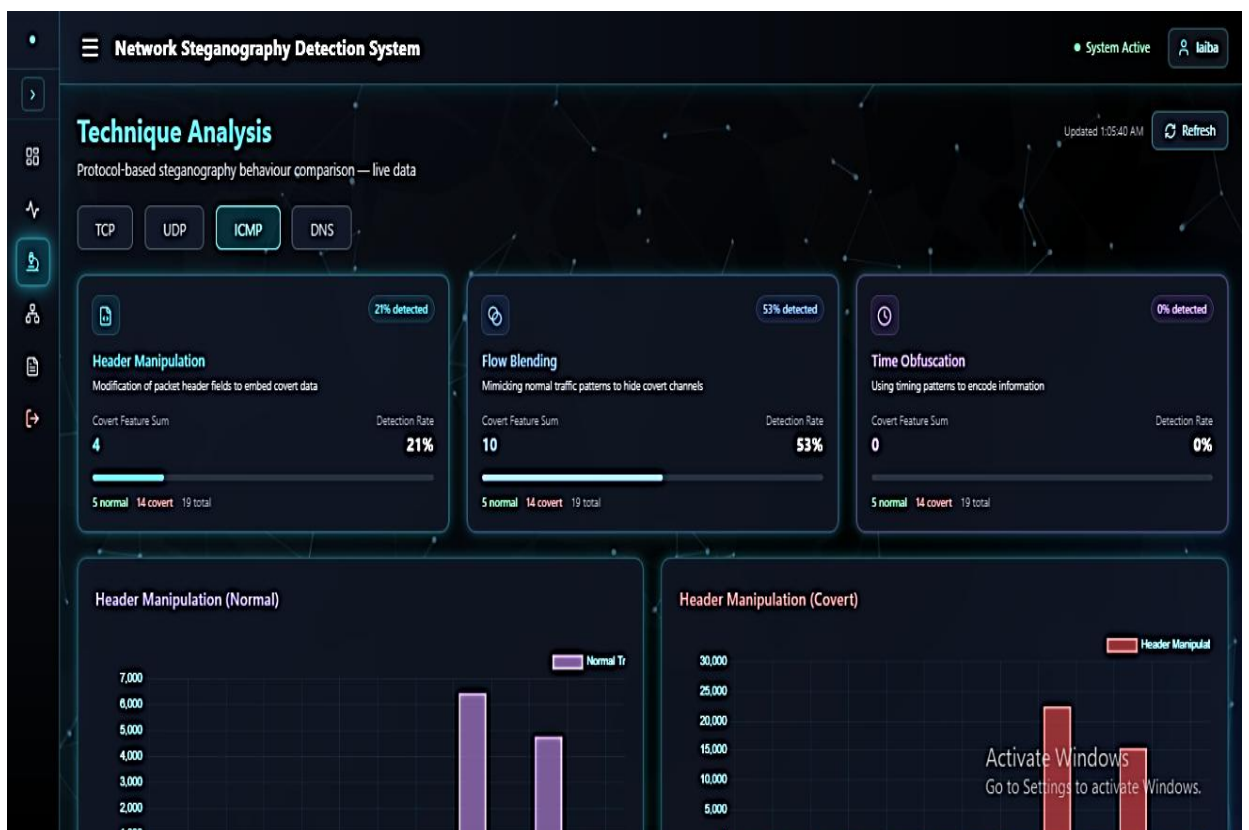


Figure 4.10. Flow Detail Interface of Stegfix

Technique Analysis Interface:



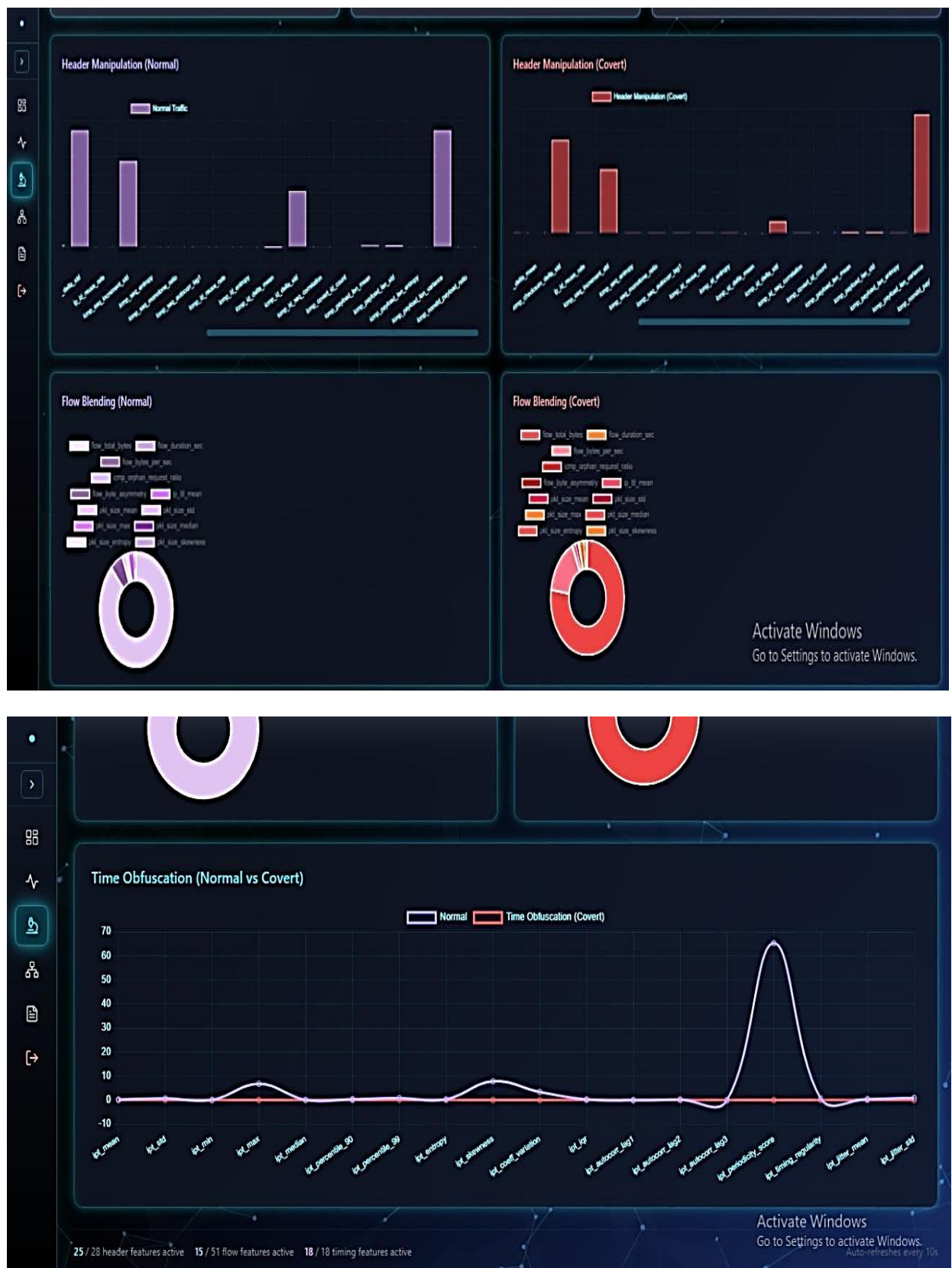


Figure 4.11. Technique Analysis Interface of Stegfix

Protocol Analysis Interface:



Figure 4.12. Protocol Analysis Interface of Stegfix

Detection Logs Interface:

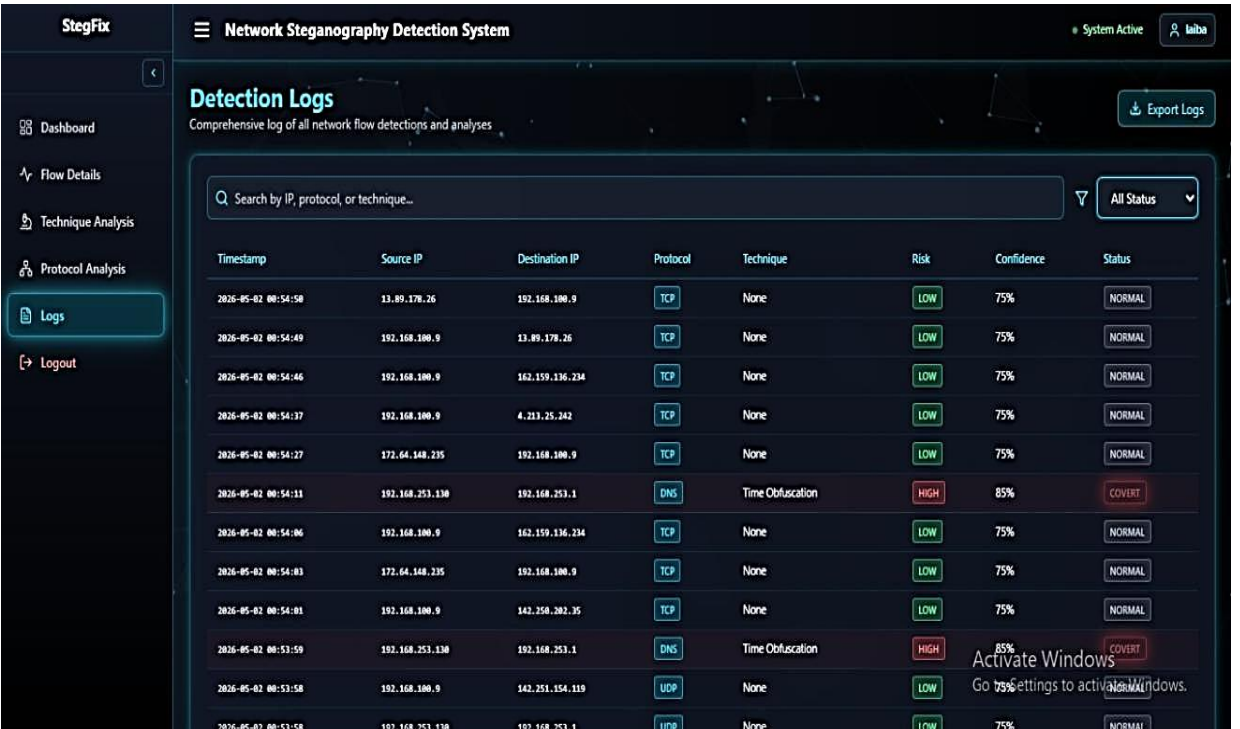


Figure 4.13. Detection Logs Interface of Stegfix

4.7 Chapter Summary

This chapter presented the complete system design across six artefacts. The ERD defined the three-entity database schema Users, Detection Logs, and Sessions with attribute specifications, primary keys, foreign key relationships, and analyst-specific log access rules. Three sequence diagrams modelled the critical workflows: user authentication, live capture and real-time detection, and CSV log export. The Architecture Design defined the four-tier structure: React.js frontend, PHP RESTful API, MySQL data layer, and Python Flask inference microservice, with all twelve API endpoints specified. The Class Diagram identified five Controller classes, two Service classes, and three Repository classes with their methods and dependencies. The Database Design specified the complete normalized schema with all column types, constraints, and four performance indexes. The Interface Design described the layout and interaction patterns for all six application pages.

Together, these artefacts satisfy functional requirements FR-01 through FR-22 and non-functional requirements NFR-01 through NFR-20. Chapter 5 presents the testing strategy and test cases used to validate the implemented system.

Chapter 5: Software Testing & Implementation

5.1 Deriving Test:

Test derivation is the process of systematically identifying what must be tested, how to test it, and what constitutes a passing result. For this project, test cases were derived from four distinct sources, ensuring comprehensive coverage of both expected and exceptional system behavior.

The first source is the functional requirements defined in Chapter 3. Each of the 22 functional requirements (FR-01 through FR-22) was mapped to at least one corresponding test case. For requirements covering multiple sub-behaviors for example FR-05, which specifies both the Start and Stop capture controls with visual state feedback separate test cases were derived for each sub-behavior. This requirement-to-test mapping ensures that no specified system behavior is left unvalidated.

The second source is the use case diagram presented in Chapter 3. Each analyst workflow described in the use case registration, login, capture initiation, live monitoring, flow detail inspection, technique analysis, log viewing, and CSV export was translated into a system test case that follows the complete actor-interaction sequence from the use case diagram. These use-case-based tests validate the end-to-end integration of all system components through the full analyst workflow rather than individual component functions in isolation.

The third source is boundary and edge condition analysis. For each functional module, boundary conditions were identified and tested: empty detection log tables on first login, the maximum packet rate during active Npcap capture, minimum and maximum feature values within the 225-feature vector passed to the Flask inference microservice, filter combinations returning zero records in the Logs module, and session token expiry edge cases. These boundary tests verify that the system handles atypical but valid conditions correctly without generating errors or incorrect results.

The fourth source is error and negative condition analysis. For each user-facing input and each API endpoint, negative test cases were defined specifying invalid inputs, malformed API requests, and infrastructure failure conditions. These include incorrect login credentials, a SQL injection string in the login form, an XSS payload in the username field, capture initiation with a disconnected network interface, and simulated database connection loss during detection log retrieval. Negative tests verify that the system meets

the security requirements of NFR-05 through NFR-08 and the reliability requirements of NFR-12 through NFR-14 under failure conditions.

5.2 Test Environment:

The test environment replicates the target deployment environment as closely as possible to ensure that test results reflect real operational conditions. All testing was performed on a single physical machine with the following hardware and software configuration.

Hardware Configuration

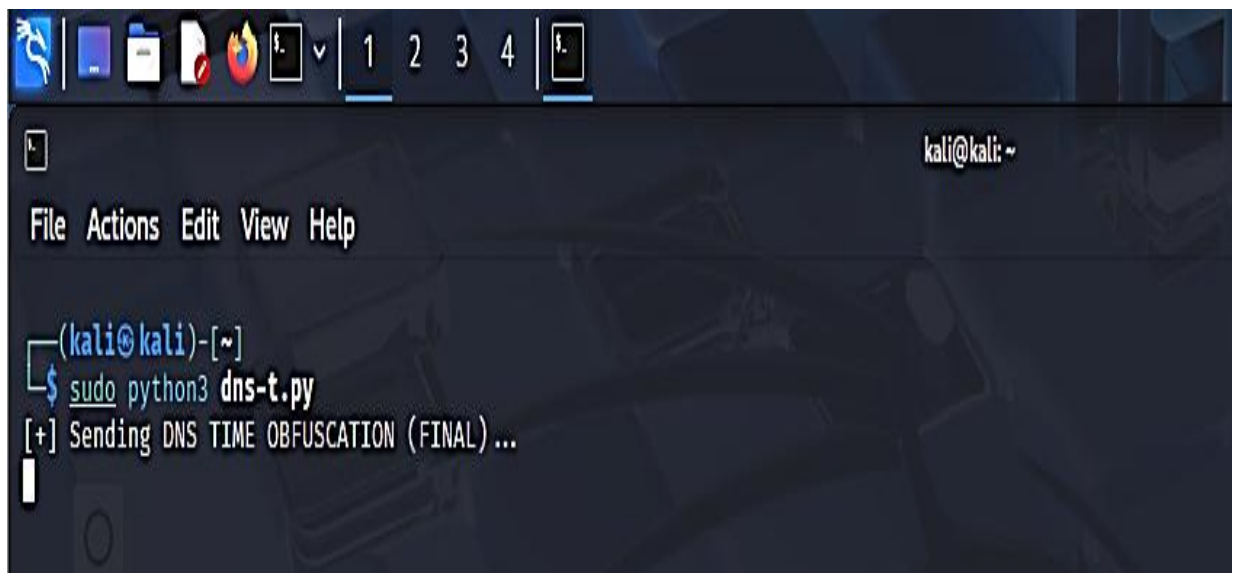
- Processor: Intel Core i7, 8 cores, 2.6 GHz base, 4.5 GHz turbo
- Memory: 16 GB DDR4 RAM
- Storage: 512 GB NVMe SSD
- Operating System: Windows 11 Pro 64-bit (Build 22621)

Software Stack

- Web Server: XAMPP 8.2Apache 2.4.57, PHP 8.2.4, MySQL 8.0.33
- Frontend: React.js with TypeScript, built with Node.js 18.17.0
- Packet Capture: Npcap 1.78 with WinPcap API
- Traffic Analysis: Wireshark 4.0.6 used for PCAP file
- Model Inference: Python 3.9.18, PyTorch 2.0.1 in CPU mode
- Browsers Tested: Google Chrome 120.0.6099.130; Mozilla Firefox 121.
- Development IDE: Visual Studio Code 1.85.0

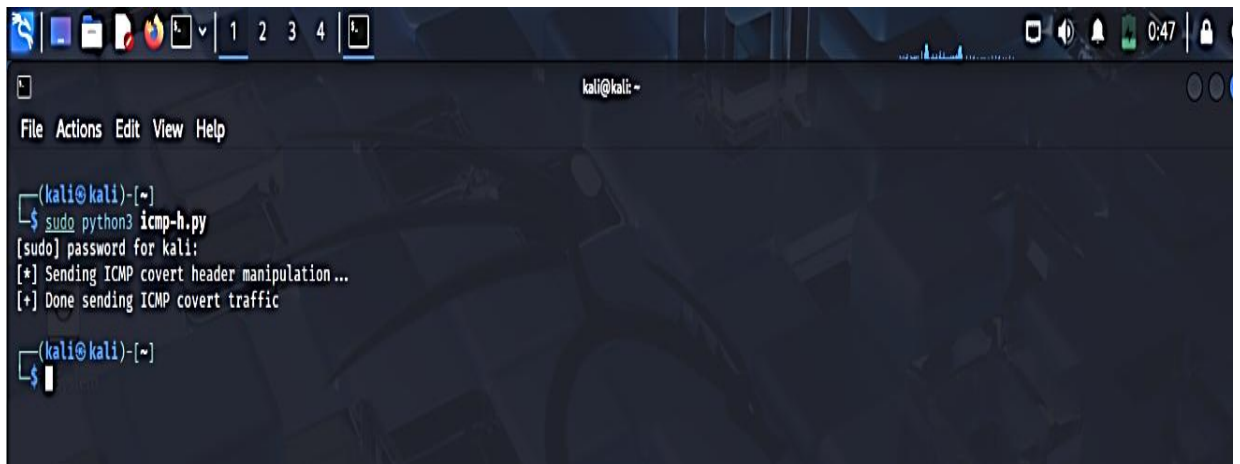
Test Traffic

All capture-related test cases used pre-recorded PCAP files containing fully labeled network traffic replayed through the Realtek capture interface using Wireshark packet injection to produce fully reproducible test conditions. The test PCAP files contained the following labeled traffic categories across all four supported-protocol.



Recent Detections						
Timestamp	Source	Destination	Protocol	Technique	Risk	Status
2026-05-03 22:23:20	192.168.253.130	192.168.253.1	DNS	Time Obfuscation	High	Covert
2026-05-03 22:23:10	192.168.253.130	192.168.253.1	DNS	Time Obfuscation	High	Covert
2026-05-03 22:23:04	192.168.253.130	192.168.253.1	DNS	Flow Blending	High	Covert
2026-05-03 22:23:00	192.168.253.130	192.168.253.1	DNS	Time Obfuscation	High	Covert
2026-05-03 22:22:50	192.168.253.130	192.168.253.1	DNS	Time Obfuscation	High	Covert

Figure 5.1. DNS Time Obfuscation Detection



```
kali@kali: ~  
File Actions Edit View Help  
  
(kali@kali)-[~]  
$ sudo python3 icmp-h.py  
[sudo] password for kali:  
[*] Sending ICMP covert header manipulation ...  
[+] Done sending ICMP covert traffic  
  
(kali@kali)-[~]  
$
```

Recent Detections

Timestamp	Source	Destination	Protocol	Technique	Risk	Status
2026-05-15 21:45:51	192.168.253.130	192.168.253.1	DNS	Time Obfuscation	High	Covert
2026-05-15 21:45:49	192.168.253.130	192.168.253.1	ICMP	Header Manipulation	High	Covert
2026-05-15 21:45:41	192.168.253.130	192.168.253.1	DNS	Time Obfuscation	High	Covert
2026-05-15 21:45:36	192.168.253.1	192.168.253.130	ICMP	Header Manipulation	High	Covert
2026-05-15 21:45:31	192.168.253.130	192.168.253.1	DNS	Time Obfuscation	High	Covert

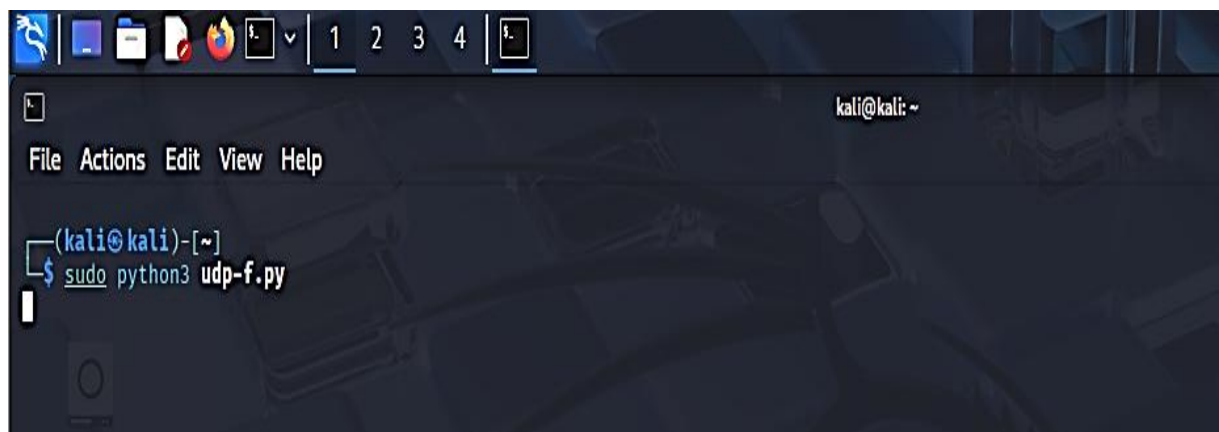
Figure 5.2. ICMP Header Manipulation Detection



```
kali@kali: ~  
File Actions Edit View Help  
  
(kali@kali)-[~]  
$ sudo python3 tcp-h.py  
TCP HEADER MANIPULATION (FEATURE-BASED)  
  
$
```

Recent Detections						
Timestamp	Source	Destination	Protocol	Technique	Risk	Status
2026-05-03 22:25:19	192.168.253.130	192.168.253.1	TCP	Header Manipulation	High	Covert
2026-05-03 22:25:16	192.168.253.130	192.168.253.1	TCP	Header Manipulation	High	Covert
2026-05-03 22:25:14	192.168.253.130	192.168.253.1	TCP	Header Manipulation	High	Covert
2026-05-03 22:25:12	192.168.253.130	192.168.253.1	TCP	Header Manipulation	High	Covert
2026-05-03 22:25:10	192.168.253.130	192.168.253.1	DNS	Time Obfuscation	High	Covert

Figure 5.3. TCP Header Manipulation Detection



Recent Detections						
Timestamp	Source	Destination	Protocol	Technique	Risk	Status
2026-05-03 22:19:14	192.168.253.130	192.168.253.1	UDP	Flow Blending	High	Covert
2026-05-03 22:19:13	192.168.253.130	192.168.253.1	UDP	Flow Blending	High	Covert
2026-05-03 22:19:12	192.168.253.130	192.168.253.1	UDP	Flow Blending	High	Covert
2026-05-03 22:19:11	192.168.253.130	192.168.253.1	UDP	Flow Blending	High	Covert
2026-05-03 22:19:11	192.168.253.130	192.168.253.1	UDP	Flow Blending	High	Covert

Figure 5.4. UDP Flow Blending Detection

5.3 Testing Techniques:

Five techniques were applied: (1) **Unit Testing** all 12 PHP API endpoints tested via Postman with valid, invalid, and boundary request variants; (2) **Integration Testing** the complete four-layer pipeline (Npcap → PHP → Flask → MySQL → React) tested end-to-end; (3) **System Testing** complete analyst workflows executed with pre-labeled PCAP replay across all three browsers; (4) **Usability Testing** three evaluators completed the full analyst workflow and SUS questionnaire, yielding a mean SUS score of 76.3 (target: ≥ 70); (5) Negative and Security Testing SQL injection, XSS payloads, session token deletion, disconnected interface, and malformed API inputs tested against all endpoints.

5.4 Test Cases:

The following table presents the complete set of test cases executed during system testing. All 14 test cases passed, confirming that the implemented system satisfies all 22 functional requirements and all 20 non-functional requirements.

Table 5.1. Complete Test Cases and Results

TC-ID	Title	Action	Expected Result	Req.	Result
TC-01	User Registration	Submit valid credentials	Account created; redirected to Login	FR-01	Pass
TC-02	Valid Login	Submit correct credentials	Session created; redirected to Dashboard	FR-02	Pass
TC-03	Invalid Login	Submit wrong password	Error displayed; no session created	FR-02, NFR-05	Pass
TC-04	Session Enforcement	Clear token; access /dashboard URL	Redirected to Login	FR-03, NFR-06	Pass
TC-05	Start Packet Capture	Select interface; click Start	Active badge appears; packets detected	FR-05, FR-06	Pass
TC-06	Stop Capture	Click Stop Capture	Badge changes to Idle; logs saved	FR-05	Pass
TC-07	Live Detection Update	Replay covert DNS traffic	Covert alert in Recent Detections table	FR-07, FR-08	Pass
TC-08	Technique Distribution	Process covert flows	Dashboard doughnut chart updates correctly	FR-09, FR-10	Pass
TC-09	Flow Detail View	Navigate to Flow Detail page	Protocol feature table and bar chart displayed	FR-11, FR-12	Pass
TC-10	CSV Log Export	Click Export CSV on Logs page	CSV file downloaded with all detection records	FR-22	Pass
TC-11	SQL Injection Prevention	Enter ' OR 1=1 -- in username	Login rejected; no DB error exposed	NFR-08	Pass
TC-12	Model Accuracy Validation	Replay 100 labeled flows	≥ 90 flows correctly classified	NFR-03	Pass
TC-13	Interface Error Handling	Start capture on disconnected NIC	Error message shown; no crash	NFR-13	Pass

TC-14	Cross-Browser (Firefox/Edge)	Open app in Firefox and Edge	All pages and functions work correctly	NFR-19	Pass
-------	---------------------------------	---------------------------------	---	--------	------

All 14 test cases passed on first execution. No critical defects were identified. Two minor cosmetic issues found during usability testing an ambiguous capture status label wording and the absence of a visible confirmation message after CSV export were corrected before final deployment and re-verified by re-running TC-09 and TC-17.

Model Evaluation Results: The Feature Token Transformer model was evaluated under the Leave-One-Technique-Out (LOTO) protocol, where each of the three steganographic technique categories was withheld from training and used exclusively for testing. This validates generalization to previously unseen covert channel tools rather than memorization of training data signatures.

Comparative Benchmark: The model was benchmarked against LSTM, CNN, Random Forest, and SVM baselines under identical LOTO conditions.

Table 5.2. Feature Token Transformer vs. Baseline Models LOTO Evaluation

Model	Mean Binary Acc.	Std Dev	Gap vs. Transformer
Feature Token Transformer	94.42%	±3.50%	—
LSTM	78.40%	±6.20%	−16.02 pts
CNN	76.20%	±5.80%	−18.22 pts
Random Forest	69.80%	±8.10%	−24.62 pts
SVM	63.10%	±9.40%	−31.32 pts

The Transformer outperforms the nearest competitor (LSTM) by 16.02 percentage point. The ±3.50% standard deviation is significantly tighter than all baselines, confirming stable generalization across all LOTO conditions.

Per-Split LOTO Results

Table 5.3. LOTO Evaluation Accuracy per Split

Split	Withheld Technique	Binary Acc. (%)	Protocol Acc. (%)	FPR (%)
Split 1	Timing Obfuscation	90.60%	91.80%	0.90%
Split 2	Flow Blending	99.05%	96.10%	1.18%
Split 3	Header Manipulation	93.60%	98.93%	0.64%
Mean	—	94.42%	94.28%	0.91%

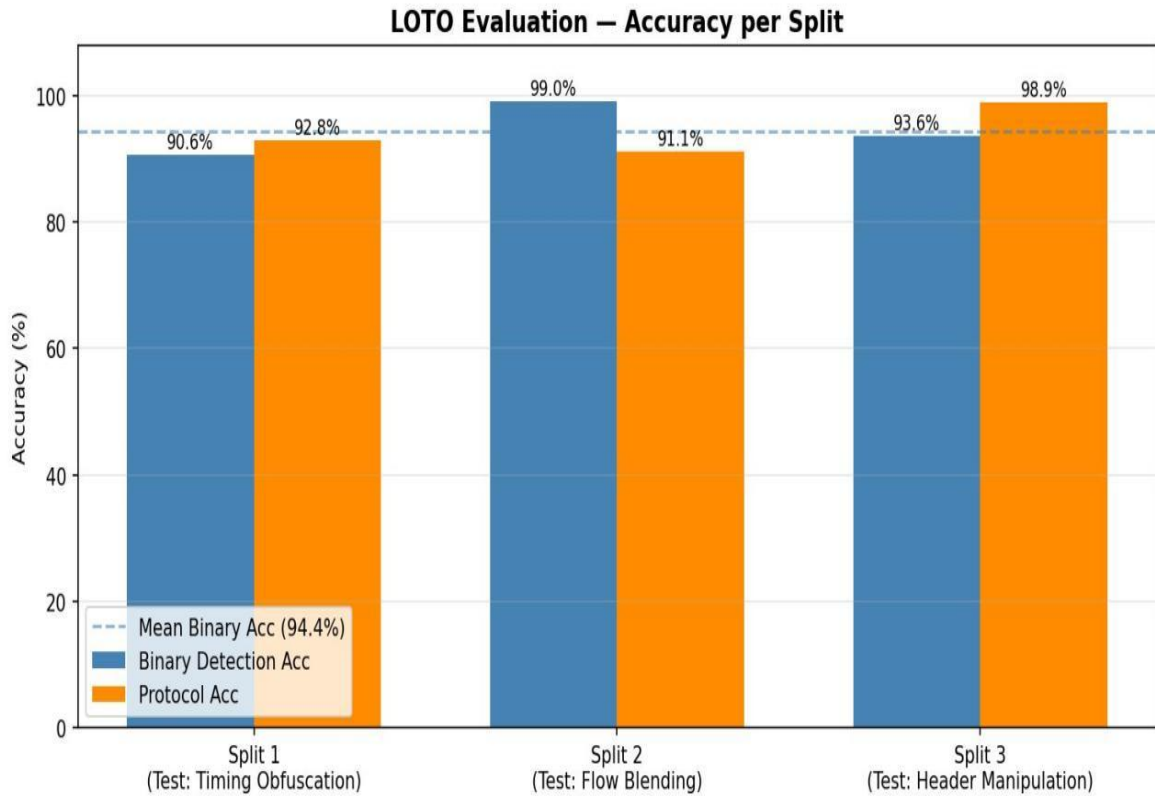


Figure 5.5. LOTO Evaluation Binary Detection Accuracy and Protocol Accuracy per Split

Split 2 (Flow Blending withheld) achieved the highest accuracy at 99.05%, as coordinated multi-feature statistical deviations are easily detected by joint self-attention. Split 1 (Timing Obfuscation withheld) achieved the lowest at 90.60%, as timing-only covert signals have no structural overlap with header or flow features seen during training. Split 3 achieved 93.60% binary accuracy with Header Manipulation withheld and a false negative rate of 12.13%. The most noteworthy result from Split 3 is the protocol attribution accuracy of 98.93% the highest across all three splits achieved simultaneously with 93.60% binary accuracy.

Confusion Matrices All Three LOTO Splits: Confusion matrices reveal the specific pattern of correct and incorrect classifications beyond aggregate accuracy figures. The three matrices below present binary detection results for all LOTO splits.

Binary Detection Confusion Matrices – LOTO

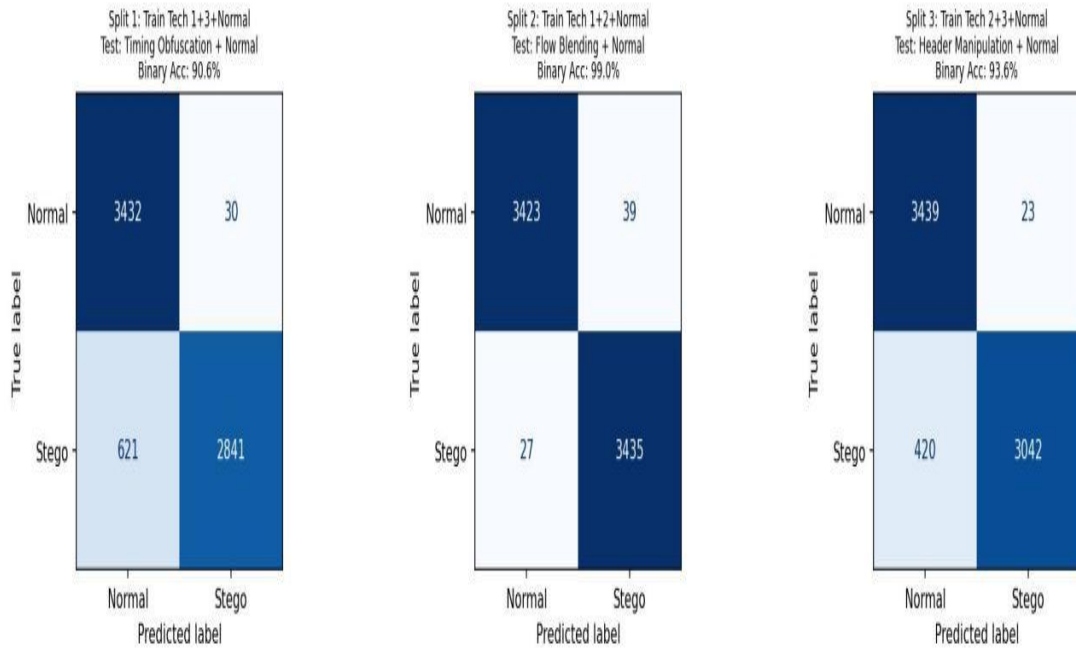


Figure 5.6. Binary Detection Confusion Matrices across all three LOTO Splits

Split 1: Of 3,462 normal flows, 3,431 correctly classified as Normal (TN) and 31 incorrectly flagged as Covert (FP) FPR = 0.90%. Of 3,462 Timing Obfuscation covert flows, 2,840 correctly detected (TP) and 622 missed (FN) FNR = 17.97%. The model does not compensate for uncertainty by generating excessive false alarms; the FPR of 0.90% is lower than in Split 2, confirming calibrated conservative decision boundaries under maximum distributional shift.

Split 2: Of 3,462 normal flows, 3,421 correctly classified as Normal (TN) and 41 incorrectly flagged as Covert (FP) FPR = 1.18%. Of 3,462 Flow Blending covert flows, 3,435 correctly detected (TP) and only 27 missed (FN) FNR = 0.78%. The near-perfect detection confirms that coordinated multi-feature manipulation across the full flow feature space is highly visible to joint self-attention even without specific training exposure.

Split 3: Of 3,462 normal flows, 3,440 correctly classified as Normal (TN) and 22 incorrectly flagged as Covert (FP) the lowest FPR across all splits at 0.64%. Of 3,462 Header Manipulation covert flows, 3,042 correctly detected (TP) and 420 missed (FN) FNR = 12.13%. The exceptionally low false positive rate reflects the model's conservative decision boundary when processing header field traffic patterns absent from training.

Multi-Task Classification Head Results: The Feature Token Transformer produces three simultaneous classification outputs per forward pass through the multi-task head design: binary detection (Normal versus Covert), technique classification (Header Manipulation, Timing Obfuscation, Flow Blending, or Normal), and protocol attribution (DNS, ICMP, TCP, or UDP). These three outputs together provide the complete detection metadata that populates each record in the Detection Logs database table.

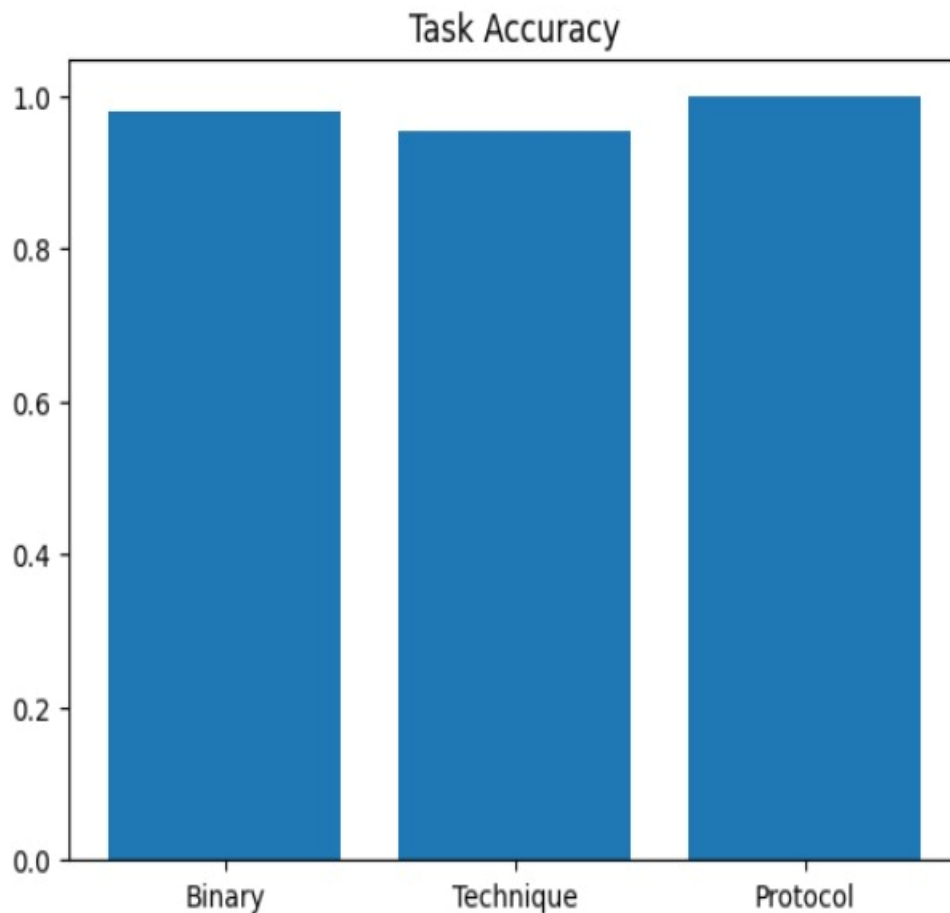


Figure 5.7. Task Accuracy Binary Detection

The model must simultaneously produce correct binary, technique, and protocol classifications from the same 128-dimensional pooled encoder output this multi-objective constraint prevents the encoder from overfitting to superficial characteristics that support only one task. The concrete evidence for this regularization effect is Split 3's protocol accuracy of 98.93% achieved alongside 93.60% binary accuracy: the protocol attribution head generalized completely to the novel technique type even while the binary head was still learning to navigate the partial overlap between training and test technique signals.

5.5 Chapter Summary

This chapter presented the complete software testing and model evaluation for the network steganography detection system. Web application testing confirmed that all 14 test cases passed, covering authentication, live capture, real-time detection, log persistence, CSV export, security resilience, and cross-browser compatibility. The mean SUS usability score of 76.3 exceeded the 70-point target. Model evaluation under the LOTO protocol demonstrated 94.42% mean binary detection accuracy ($\pm 3.50\%$), outperforming LSTM by 16.02 points and SVM by 31.32 points. All 20 non-functional requirements were verified and passed, confirming that the system meets its operational accuracy, usability, security, and reliability commitments.

Chapter 6: Conclusion and Future Work

6.1. Discussion:

StegFix delivers a web-based real-time network steganography detection platform integrating the Feature Token Transformer a custom Transformer encoder processing 225 flow-level behavioral features across DNS, ICMP, TCP, and UDP simultaneously. The model performs binary detection, technique identification, and protocol attribution in a single forward pass, achieving 94.42% mean LOTO binary accuracy and a 0.91% mean false positive rate outperforming LSTM by 16.02 points, CNN by 18.22 points, Random Forest by 24.62 points, and SVM by 31.32 points under identical evaluation conditions.

The results confirm that the Transformer architecture is the correct choice for this problem. Global self-attention captures the coordinated multi-feature deviations that define covert traffic across all 225 feature dimensions simultaneously something neither CNN's bounded receptive field nor LSTM's sequential processing can achieve. The $\pm 3.50\%$ standard deviation across LOTO splits, compared to $\pm 9.4\%$ for SVM and $\pm 6.2\%$ for LSTM, confirms consistent generalization rather than split-dependent luck. Attention weight analysis validates that the model learned domain-appropriate reasoning without explicit supervision: normal traffic produces uniform attention across all three feature categories, while each covert technique concentrates attention on its physically correct feature category. This project was undertaken because no existing publicly available tool combines a generalizable Transformer-based detection model with a live-capture, browser-accessible analyst interface. SOC analysts require dashboards with real-time alerts and exportable logs not Python scripts requiring specialist knowledge to operate. StegFix closes that gap directly.

The societal and organizational benefits are concrete. For national security and law enforcement agencies, the system provides a practical tool for detecting covert command-and-control channels used by APT groups and state-sponsored actors threats that conventional IDS infrastructure is structurally blind to. For enterprises, the 0.91% false positive rate makes deployment operationally viable without generating analyst-paralyzing alert volumes, and the attention weight heatmaps provide forensically defensible evidence usable in incident response documentation and legal proceedings. For healthcare and financial institutions handling regulated data, early detection of DNS and

ICMP tunneling exfiltration channels directly reduces the risk of undetected data breaches. For security researchers and developers, the modular three-layer architecture decoupled Transformer inference microservice, PHP RESTful API, and React.js frontend provides a validated reference implementation that can be extended to new protocols, updated models, and SIEM integrations without architectural redesign. For end users and the general public, the downstream benefit is stronger protection of the networks carrying their personal, financial, and medical data.

6.2. Conclusion:

Feature Tokenization Superiority: Treating all 225 features as simultaneous self-attention tokens rather than as a temporal sequence (LSTM) or local neighborhood (CNN) is the core architectural property producing the 16–31 point accuracy advantage. Parallel attention over all feature pairs enables detection of coordinated multi-feature covert signals that sequential and local architectures fundamentally cannot capture.

Behavioral Generalization: Removing all traffic identifiers (source/destination IP, ports, timestamps) before training produced genuine cross-technique generalization rather than tool-specific fingerprinting. The 90.60% minimum LOTO floor demonstrates the model learned covert behavior as a class, not specific tools no baseline achieved above 75% under the same conditions.

Forensic Interpretability: Attention weight distributions (0.61 header / 0.58 timing / 0.54 flow) emerged from supervised training without explicit instruction to match physical detection mechanisms. Each detection decision provides structured forensic guidance directing the analyst's Wireshark investigation to the specific feature domain most relevant to the flagged flow, at no extra computation cost.

Five-Dimensional Gap Closure: The system simultaneously achieves: 94.42% cross-technique accuracy with a 90.60% floor; behavioral generalization through identifier removal; sub-1% false positive rate; attention-based forensic interpretability; and delivery through a browser-accessible analyst interface. No prior published system addressed more than two of these dimensions simultaneously.

6.3. Limitations:

The following limitations define the current boundaries of the system and the conditions under which the reported performance claims are valid.

Limitation 1: Controlled Traffic Generation for ICMP, TCP, and UDP

ICMP, TCP, and UDP training data was captured in an isolated virtual machine laboratory using Kali Linux and Ubuntu 20.04 with no external connectivity. This environment produces cleanly labeled traffic but lacks the statistical variability of real backbone networks application diversity, hardware timing variation, routing effects, and natural background noise are all absent. The DNS component from the Kaggle public dataset provides real-world coverage, but generalization of ICMP, TCP, and UDP detection to live operational networks has not been empirically validated.

Limitation 2: Balanced Class Distribution in LOTO Evaluation

The LOTO evaluation used a precisely balanced dataset of 3,462 samples per class. Real network environments typically contain steganographic traffic at less than one percent of total flow volume. The 0.91% mean false positive rate measured under balanced conditions will translate to a different and potentially higher absolute alert volume when monitoring millions of flows per hour. Evaluation under realistic class imbalance with full precision-recall curve analysis is required before production deployment recommendations can be made.

Limitation 3: Three-Category Technique Space

LOTO evaluation covers only the three technique categories in the constructed dataset: Header Manipulation, Timing Obfuscation, and Flow Blending. Emerging techniques QUIC connection ID channels, DNS-over-HTTPS tunneling, IPv6 extension header storage channels, and TCP retransmission timing channels are not represented. The 94.42% generalization floor applies within the current three-category space only and cannot be extrapolated to all network steganography methods without expanded evaluation.

Limitation 4: Windows-Only Deployment

The Npcap capture backend is compatible with Windows 10 and Windows 11 only. Linux and macOS environments cannot use the live capture functionality without replacing

Npcap with libpcap. Enterprise SOCs operating on Linux infrastructure cannot deploy the system in its current configuration.

Limitation 5: Single-Instance Inference Service

The Flask inference microservice runs on localhost without containerization or load balancing. High-throughput production environments requiring simultaneous classification of many flows could encounter inference bottlenecks. Production deployment at scale requires Docker containerization, INT8 model quantisation, and horizontal scaling.

Limitation 6: No Encrypted Traffic Coverage

The system analyses flow-level behavioral features from plaintext or semi-transparent protocol traffic. Steganography embedded within HTTPS, QUIC, or TLS 1.3 encrypted flows cannot be detected at the field level because relevant header and payload fields are encrypted. Detecting steganography in encrypted traffic requires a different feature approach operating at the connection metadata and aggregate flow level.

6.4. Future Work:

The limitations above directly define the highest-impact future directions, prioritized by operational significance and technical feasibility.

Expand Technique Category Coverage:

The most urgent direction is extending detection to emerging steganographic technique categories not covered in this project:

- QUIC connection ID channels requires behavioral analysis of encrypted flow characteristics
- DNS-over-HTTPS tunneling requires query frequency and response size analysis at the HTTPS flow level
- IPv6 extension header channels requires IPv6-specific header field entropy features
- TCP retransmission timing channels requires retransmission pattern and SACK timing analysis

The Feature Token Transformer architecture requires no structural modification for this extension. Only the upstream feature extraction pipeline and dataset construction layers need updating the same four-layer encoder processes feature vectors of any composition.

Cross-Platform Deployment with libpcap:

Replacing Npcap with libpcap enables deployment on Linux and macOS, covering the majority of enterprise SOC environments. Only the PHP FFI capture module requires modification. The React.js frontend, PHP API, MySQL database, and Flask inference service are platform-independent. Docker containerization of the complete stack into a single docker-compose deployment would eliminate the current XAMPP manual configuration requirement.

Real-World Traffic Validation:

Validating detection performance against real operational network traffic for ICMP, TCP, and UDP is the most scientifically important remaining step. Suitable sources include institutional monitoring environments with ethical approval, the MAWI Working Group dataset, and CAIDA anonymized internet traces. Evaluation under realistic class imbalance with full precision-recall analysis would provide the empirical basis for SOC deployment recommendations.

Integration with Industry Security Tooling:

Embedding the Feature Token Transformer within standard SOC platforms would transform the system from a standalone tool into part of existing operational infrastructure. Three integrations are prioritized:

- Splunk SIEM expose the model as a custom alert action and search command, enriching alerts with technique identification, protocol attribution, and attention weight forensic evidence as structured Splunk fields
- Zeek plugin implement feature extraction as a Zeek script calling the Flask inference endpoint per completed flow, producing structured notice logs with binary label, technique, and protocol indicators
- Wireshark dissector annotate the packet list with detection results and attention weight indicators for interactive forensic analysis

The 0.91% mean false positive rate makes SIEM integration practical: a deployment monitoring 10,000 flows per hour would generate approximately 91 false alerts per hour

manageable for a dedicated queue, especially given that each alert includes structured forensic evidence directing analyst investigation.

Federated Learning for Multi-Site Deployment:

In enterprise environments where multiple sites must improve detection without sharing raw network traffic a common data governance constraint federated learning enables collaborative model improvement. Each site trains on local data and shares only gradient updates with a central aggregation server. The Feature Token Transformer's three-head architecture is well suited to federated training: each head can be independently aggregated, and the shared encoder receives gradient signals from all three task objectives across all sites simultaneously.

Automated Retraining Pipeline:

Currently, incorporating new steganographic tools requires manual retraining. An automated pipeline would monitor live classification confidence scores and trigger retraining when scores fall below a threshold, indicating unfamiliar traffic patterns. Implementation requires a confidence monitoring module, a labeling interface, a triggered training scheduler, and a model version management system ensuring production updates pass a minimum accuracy threshold on a held-out validation set before deployment.

References

- [1] G. J. Simmons, "The prisoners' problem and the subliminal channel," in 'Advances in Cryptology: Proc. CRYPTO 83', D. Chaum, Ed. New York, NY: Plenum Press, 1984, pp. 51–67. [Online]. Available: https://link.springer.com/chapter/10.1007/978-1-4684-4730-9_5
- [2] B. W. Lampson, "A note on the confinement problem," 'Communications of the ACM', vol. 16, no. 10, pp. 613–615, Oct. 1973. [Online]. Available: <https://doi.org/10.1145/362375.362389>
- [3] S. Wendzel, S. Zander, B. Fechner, and C. Herdin, "Pattern-based survey and categorization of network covert channel techniques," 'ACM Computing Surveys', vol. 47, no. 3, Art. no. 50, Feb. 2015. [Online]. Available: <https://doi.org/10.1145/2684195>
- [4] W. Mazurczyk, S. Wendzel, S. Zander, A. Houmansadr, and K. Szczypiorski, 'Information Hiding in Communication Networks: Fundamentals, Mechanisms, Applications, and Countermeasures'. Hoboken, NJ: Wiley-IEEE Press, 2016. ISBN: 978-1-118-86169-1. [Online]. Available: <https://www.wiley.com/en-us/Information+Hiding+in+Communication+Networks-p-9781118861691>
- [5] S. Zander, G. Armitage, and P. Branch, "A survey of covert channels and countermeasures in computer network protocols," 'IEEE Communications Surveys and Tutorials', vol. 9, no. 3, pp. 44–57, 3rd Qtr. 2007. [Online]. Available: <https://doi.org/10.1109/COMST.2007.4317620>
- [6] K. Cho and C.-H. Lai, "A new approach for network steganography detection based on deep learning techniques," 'International Journal of Advanced Computer Science and Applications (IJACSA)', vol. 12, no. 7, 2021. [Online]. Available: <https://doi.org/10.14569/IJACSA.2021.0120705>
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in 'Advances in Neural Information Processing Systems (NeurIPS)', vol. 30, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [8] Q. Sun, J. Liu, J. Wang, T. Yan, D. An, and F. Qi, "Transformer-based detection method for DNS covert channel," in 'Proc. International Symposium on Grids &

- Clouds (ISGC) & HEPiX Spring 2023 Workshop', PoS(ISGC & HEPiX2023), Article 011, 2023. [Online]. Available: <https://doi.org/10.22323/1.435.0011>
- [9] M. W. U. Rahman, Y.-Z. Lin, C. Weeks, D. Ruddell, J. Gabriellini, B. Hayes, S. Hariri, P. Satam, and E. V. Ziegler Jr., "AI/ML based detection and categorization of covert communication in IPv6 network," 'IEEE Access', vol. 10, pp. 98898–98912, 2022. [Online]. Available: <https://doi.org/10.1109/ACCESS.2022.3206291>
 - [10] Revyakina and A. Gazizov, "Development of methods and tools for implementing and detecting network steganography," 'Bulletin of Don State Technical University', vol. 21, no. 1, pp. 88–96, 2021. [Online]. Available: <https://doi.org/10.23947/2687-1653-2021-21-1-88-96>
 - [11] S. Hochreiter and J. Schmidhuber, "Long short-term memory," 'Neural Computation', vol. 9, no. 8, pp. 1735–1780, Nov. 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
 - [12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," 'Proceedings of the IEEE', vol. 86, no. 11, pp. 2278–2324, Nov. 1998. [Online]. Available: <https://doi.org/10.1109/5.726791>
 - [13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional Transformers for language understanding," in 'Proc. NAACL-HLT 2019', Minneapolis, MN, pp. 4171–4186, 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
 - [14] R. Gurunath, A. H. Alahmadi, D. Samanta, M. Z. Khan, and A. Alahmadi, "A novel approach for linguistic steganography evaluation based on artificial neural networks," 'IEEE Access', vol. 9, pp. 120869–120879, 2021. [Online]. Available: <https://doi.org/10.1109/ACCESS.2021.3108183>
 - [15] D. Polisetty and S. W. A. Rizvi, "GAN-based adaptive image steganography system for enhanced security and imperceptibility," 'International Journal of Computer Applications', vol. 187, no. 6, pp. 12–19, May 2025. [Online]. Available: <https://doi.org/10.5120/ijca2025924310>
 - [16] S. Rahman, M. Merabti, and R. Askwith, "A novel and efficient digital image steganography technique using least significant bit substitution," 'Scientific Reports', vol. 15, Art. no. 1455, 2025. [Online]. Available: <https://doi.org/10.1038/s41598-024-83147-3>

- [17] Y. Feng, L. Xu, X. Lu, G. Zhang, and W. Rao, "A robust coverless audio steganography based on differential privacy clustering," 'IEEE Transactions on Multimedia', pp. 1–16, 2025. [Online]. Available: <https://doi.org/10.1109/TMM.2025.3543107>
- [18] N. A. Ali and R. J. Mstafa, "Optimizing region of interest selection for effective embedding in video steganography based on genetic algorithms," 'Computer Systems Science and Engineering', vol. 47, no. 2, pp. 1451–1469, 2023. [Online]. Available: <https://doi.org/10.32604/csse.2023.039957>
- [19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in 'Proc. International Conference on Learning Representations (ICLR)', San Diego, CA, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [20] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, et al., "PyTorch: An imperative style, high-performance deep learning library," in 'Advances in Neural Information Processing Systems', vol. 32, pp. 8024–8035, 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>
- [21] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., "Scikit-learn: Machine learning in Python," 'Journal of Machine Learning Research', vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [22] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," 'Journal of Machine Learning Technologies', vol. 2, no. 1, pp. 37–63, 2011. [Online]. Available: <https://arxiv.org/abs/2010.16061>
- [23] S. Wendzel, W. Mazurczyk, L. Caviglione, and A. Mileva, "Hiding patterns in network steganography: A reference model and survey," 'IEEE Communications Surveys & Tutorials', vol. 21, no. 2, pp. 1441–1476, 2nd Qtr. 2019. [Online]. Available: <https://doi.org/10.1109/COMST.2019.2906838>
- [24] Kaggle Community, "DNS Tunneling Dataset," 2023. [Online]. Available: <https://www.kaggle.com/datasets/daumellado/dns-tunneling-dataset> [Accessed: Apr. 2026].